

DEAKIN UNIVERSITY
Faculty of Science, Engineering and Built Environment
School of IT



Reproduction and Extension of DASMcC: Multi-Class Cardiovascular Disease Prediction from 12-Lead ECG

SIT307 – Machine Learning

Lecturer: A/Prof. Kewen Liao

Student Name: Bao Minh Tran

Student ID: s224236373

Email: s224236373@deakin.edu.au

September 20th, 2026

Abstract

This report aims to study the application of Machine Learning in Multi-class Cardiovascular Disease Prediction based on 12-lead ECG. We shall revisit a prior work on this application in Part 1, criticise their work is of poor quality and from evidence collected within this report, we claim that the results are not reproducible. Therefore, in Part 2, we propose our solution to the same dataset, and observe more promising results with and without the use of DL feature extractors. Empirically, we claim that, tree-based models perform relatively well on high-dimensional data, at an expense of computational complexity. The results presented in this report can be verified at [this repository](#).

Contents

1	Introduction and Related Work	2
2	Dataset	4
Part 1: Reproduction of Published Results		8
3	Reproduction Methodology and Protocol	8
4	Reproduction Results and Critical Analysis	11
4.1	Reproduced versus Published Results	11
4.2	Critical Analysis of Discrepancies	13
4.3	Ablation study	15
Part 2: Proposed Machine Learning Solution		20
5	Motivation	20
6	Proposed method	20
7	Results and Comparative Analysis	23
8	Ablation study: Full vs. Compressed feature vectors	26
9	Conclusion and Discussion	29

1 Introduction and Related Work

According to World Health Organisation (WHO) [1], cardiovascular disease (CVDs) is one of the urgent and primary cause of global deaths. It was estimated 19.8 million people died from CVDs, accounting for roughly 32% of all reported deaths globally, and approximately 85% were diagnosed with strokes and heart attacks. Over three quarters of CVD-related deaths were reported in low- to middle-income nations. Evidently, during 1990 and 2022, as surveyed by [2, 3], the mortality rate due to CVDs witnessed significant drops; however, in developing countries, improvement were gained but insignificant, e.g. Central Latin America, South East Asia, North Africa and Middle East. Even worse, South Asia and almost African nations suffered from the rise of CVDs-diagnosed deaths.

Besides maintaining a healthy lifestyle by managing behavioural and environmental risk factors, e.g. tobacco use, alcohol consumption, excessive use of salt, sugar and fats, physical inactivity and air pollution [4, 5]. It is of highly demanding to develop early symptoms of CVDs detectors, so that medical inference can begin as early as possible to mitigate the adverse impacts [1]. Several medical statistics can be leveraged to conduct health diagnosis, e.g. phonocardiogram (PCG) [6], photoplethysmography (PPG) [7], invasive techniques (i.e. implanting biosensors to patients' body) [8], clinical, demographic features and physical examinations [9, 10, 11, 12, 13, 14, 15, 16]. Nonetheless, in this article, we shall discuss applications of machine learning techniques for CVD early detection using Electrocardiography (ECG), which is a non-invasive and cost-effective tool [17, 18, 19]. Before reviewing state-of-the-art (SOTA) approaches and results, let us offer some background knowledge about ECG below, fully adopted from [20], which are essentially important to extract meaningful features for modelling pipeline.

The electrocardiogram or electrocardiography (ECG or EKG) is a time-series visualisation of cardiac electrical activity, offering the so-called “time-voltage chart of the heartbeat”. ECGs can be produced by attaching sensors, also known as electrodes, on different body parts, to record cardiac electrical currents. In practices, different number of locations are chosen, which produce different types of what-called ECG Leads. In this work, we shall consider the 12-lead ECG, which is believed as the gold standard for clinical practices, though smaller number of leads, e.g. single-lead and three-lead, can be used and were studied in [21, 22, 23]. An illustration of electrode placements for a standard 12-lead ECG is shown as follows.

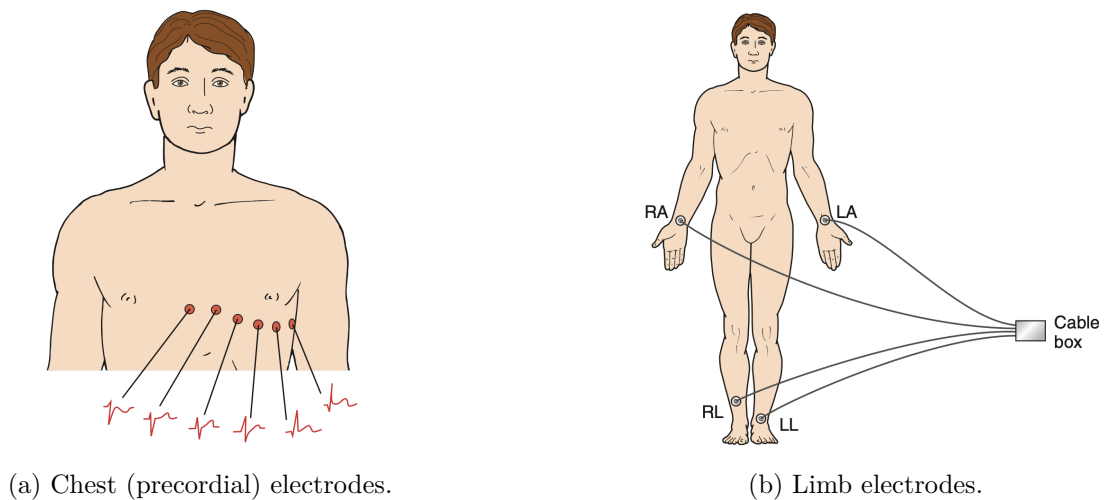


Figure 1: Electrode placement for a standard 12-lead ECG recording, adopted from [20]. (a) The six chest electrodes V_1 – V_6 . (b) The four limb electrodes: right arm (RA), left arm (LA), left leg (LL) and right leg (RL).

The 12 leads are: V_1 – V_6 + Lead I, II, III + aVR, aVL, aVF, where [20, 24] define:

$$\begin{aligned} \text{Lead I} &= \text{LA} - \text{RA}, & \text{Lead II} &= \text{LL} - \text{RA}, & \text{Lead III} &= \text{LL} - \text{LA}, \\ \text{aVR} &= -\frac{\text{Lead I} + \text{Lead II}}{2}, & \text{aVF} &= \frac{\text{Lead II} + \text{Lead III}}{2}, & \text{aVL} &= -(\text{aVR} + \text{aVF}). \end{aligned} \quad (1)$$

Since the ECG is a time–voltage chart, it is a wave–like charts, comprising of different waveforms, which we shall introduce next.

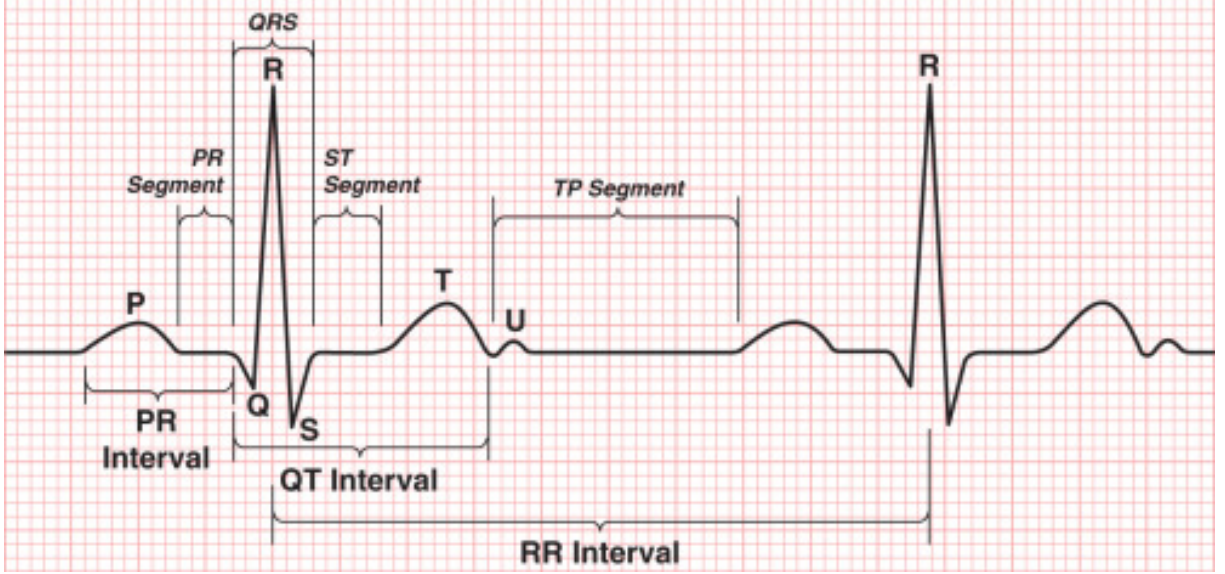


Figure 2: The characteristic waveforms, segments and intervals of a single cardiac cycle on an ECG trace, adopted from [20]. The P wave, the QRS complex, and the T wave, occasionally followed by a small U wave. The PR and QT intervals, the PR, ST and TP segments, and the RR interval are the timing measurements most commonly used in clinical interpretation.

The PTB-XL dataset consists of 12 temporal waveform charts representing 12 above–mentioned leads, each of which is characterised by the peaks, intervals and segments as demonstrated, further details will be presented in **Sec. 2**.

Due to its abundant source of informative signals yet demanding advanced expertise to comprehend and undoubtedly subtle patterns may be missed by doctors, AI–based systems has become considerably important to detect patterns in ECGs [18].

Automated ECG interpretation has progressed through two broad phases. Classical pipelines first delineate the waveform of **Fig. 2**, extract hand–crafted descriptors, i.e. wave amplitudes, PR/QT/RR durations, heart–rate variability statistics and spectral or wavelet coefficients, and pass them to shallow classifiers such as support vector machines, random forests or gradient–boosted trees. The DASMCC classifier in[17], which this report reproduces, is a recent representative of this family, it augments an imbalanced multi–class dataset using SMOTE before training tree–based ensembles on time–series features. Such pipelines remain attractive because their inputs are clinically interpretable and their computational footprint is modest.

The prevailing SOTA, by contrast, applies end–to–end deep learning directly to the raw sampled waveform, dispensing with explicit feature engineering. [25] trained a 34–layer **ResNet** on more than 90,000 single–lead ambulatory recordings and matched or exceeded an averaged cardiologist committee across twelve rhythm classes. After that, [26] scaled the approach to 12–lead

data, training on over two million exams from a Brazilian telehealth network and reporting F_1 scores above 80% at specificity above 99% for six abnormality classes. Deep models can more-over surface conditions that are not visually apparent to a human reader, e.g. [27] identified asymptomatic left-ventricular systolic dysfunction from a standard 12-lead trace with an AUC near 0.93, a capability surveyed more broadly by [18].

Because these headline results rest on private institutional cohorts, they are difficult to compare directly. The release of PTB-XL [28] supplied a large, public, multi-label benchmark. The accompanying study [19] established modern 1D-CNN architectures of the ResNet and Inception families attain a macro-averaged AUC of roughly 0.93 and consistently outperform recurrent and feature-based baselines, a ranking corroborated by [29].

The gap DASMCC targets is therefore one of deployability rather than raw accuracy. [17] argued that models trained end-to-end on millions of raw traces are too heavy for IoT-based smart healthcare framework, and offer instead a lightweight, real-time classifier driven by only sixteen time-domain features of the 12-lead signal.

The contributions of this report are:

- Critical analysis of the presented [17] vs. reproduced results.
- Proposal of our ML solution to the research problem being discussed.
- Empirical benchmark on the effect of dimensionality in ML classifiers.

Throughout this report, we shall refer to [17], which is the paper we are attempting to reproduce, as original paper.

The remainder of this report is organised as follows. In **Sec. 2**, we conduct an Exploratory Data Analysis (EDA) on PTB-XL dataset. **Part 1** presents approach and the reproduced results, along with certain critical reflections on any discrepancies identified between our and their results. **Part 2** is based on the evidence in the prior part, we argue and propose another ML solution and benchmark against their results.

2 Dataset

While conducting the experiment, we noticed some deviations with the reported dataset, and we include some basic exploratory data analysis (EDA) to offer a comprehensive overview of the dataset. The PTB-XL dataset provides us with `records100`, a downsampled version of `records500`; however, for sake of reproducibility and comparison fairness, we confine ourselves to `records100`. Furthermore, for sake of memory usage and computing purposes, we shall reduce to data from `float64` to `float32`. Also, this section serves as an agreement on the dataset used in the modelling pipeline to compare between the reproduced and our results. Firstly, let us discuss the target classes or diagnostic labels/classes, and the ECG signals thereafter. Details presented herein is extracted from the `eda.ipynb` in our repository.

For the diagnostic labels, they annotated the dataset with 24 subclasses and further grouped into 5 superclasses, by the help of at most two cardiologists. An illustration is given as below.

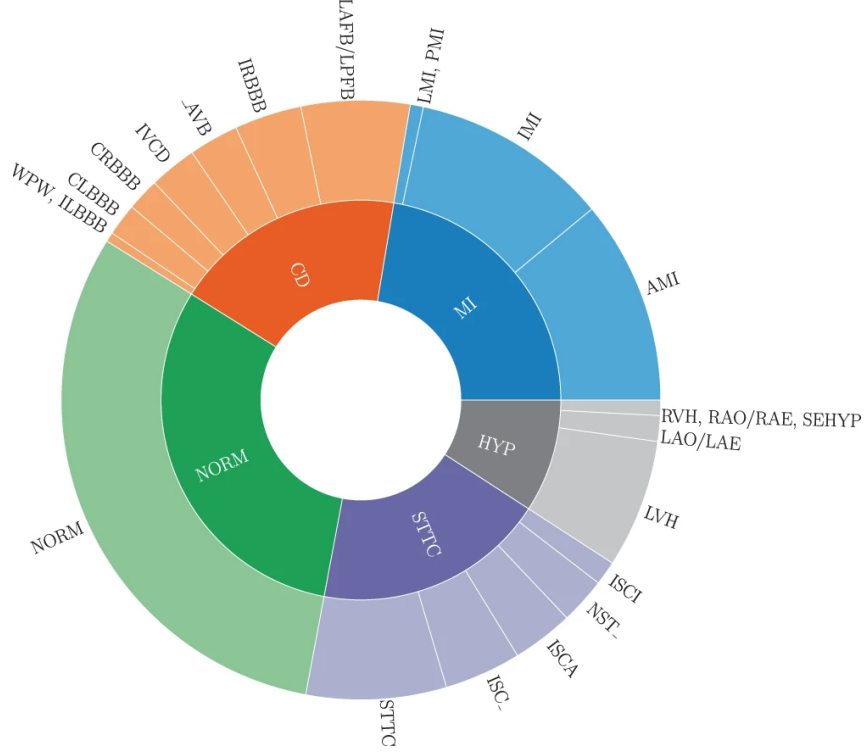


Figure 3: Graphical summary of superclasses and their subclasses, adopted from [28].

Let us first present the distribution of data w.r.t the diagnostic superclasses. We omit the description of what superclass acronyms stand for, as one can work without knowing.

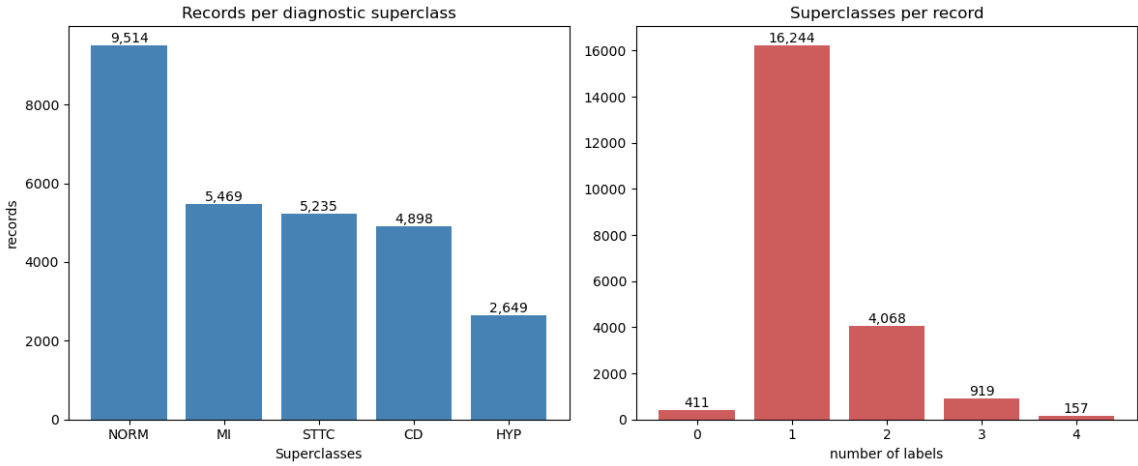


Figure 4: Distribution of the diagnostic superclasses over the **records100** subset of PTB-XL. **Left:** the number of records carrying each superclass. **Right:** the number of superclasses assigned to each record.

As described in the original paper, we are working with a multiclass, not multilabel, classification task, and thus, they only considered records annotated with one label and dropped all recorded without annotations. For brevity, we shall refer to 1-label (multiclass) dataset as “dataset”, unless stated otherwise. We present data distribution as below.

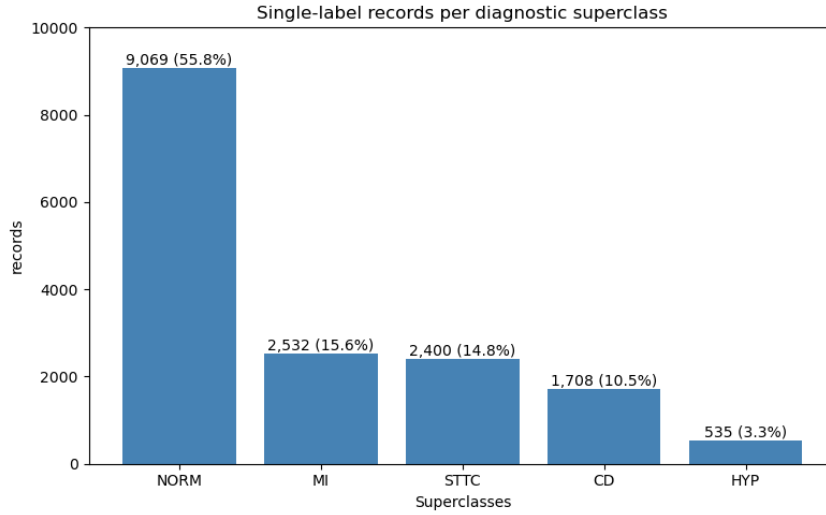


Figure 5: Distribution of the single diagnostic superclass over the **records100** subset of PTB-XL

Next, let us investigate the ECG records. In brief, each ECG record has a shape of (1000×12) , where 1000 represents the number of samples within 10 seconds, and 12 implies the number of leads used in the clinical study. Let us illustrate the first ECG record in the dataset as follows.

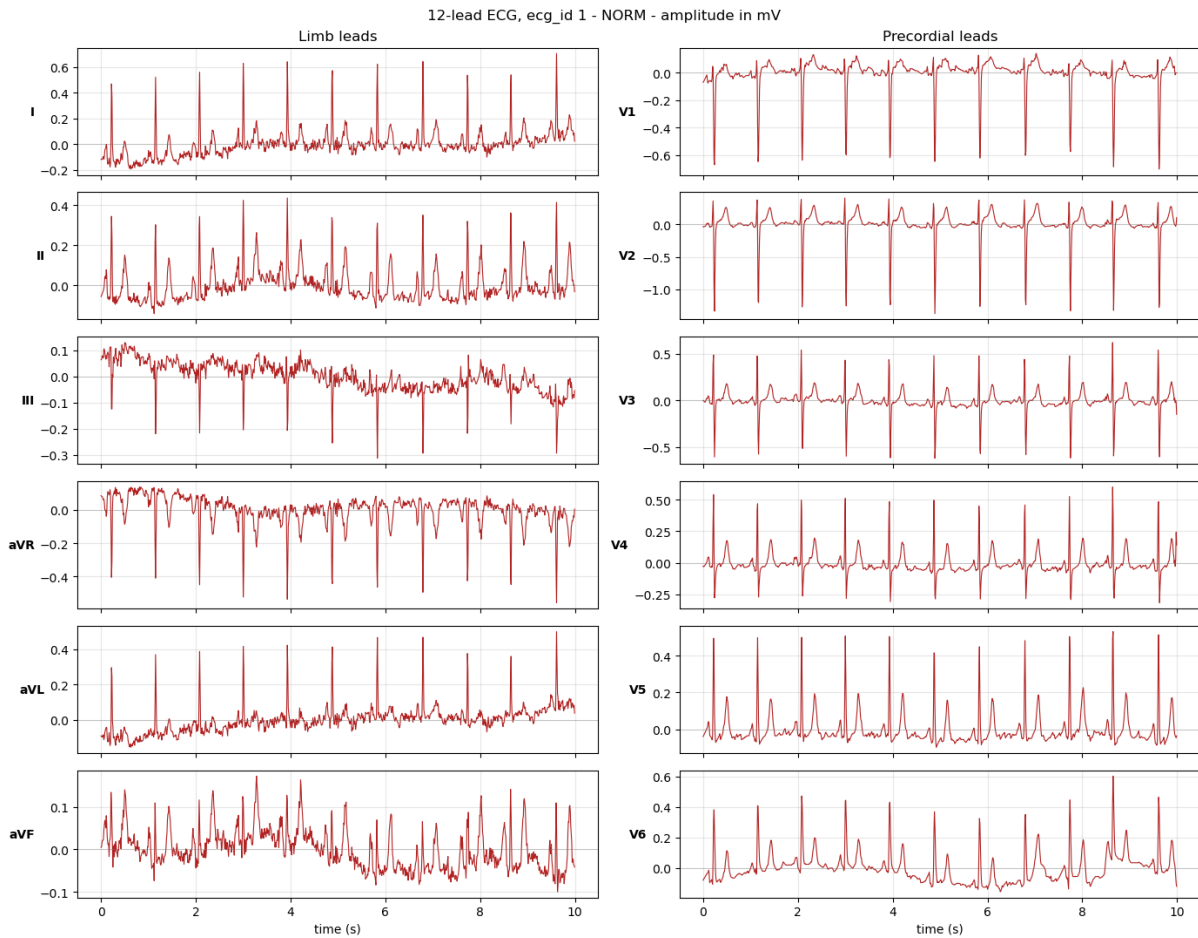


Figure 6: 12-lead ECG visualisation.

Next, let us verify if there are missing values in the dataset. We verify if there are any **NaN/Inf** in the records, also, if any ECG record shows zero variance, this may be subject to sensor errors, and treated as missing values. Let us demonstrate the distribution of the ECG signals w.r.t all 12 leads as follows.

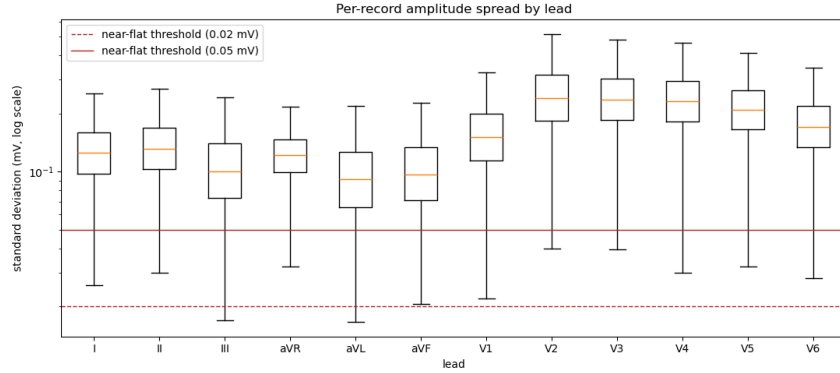


Figure 7: Per-record amplitude standard deviation.

More thoroughly, there are no missing values presented; still, many ECG records comprise of low-variance signals, which might be recorded from CVD patients.

Table 1: Near-flat leads and the records containing them, at increasing thresholds on the per-lead amplitude standard deviation σ .

Threshold	Near-flat leads	Affected records
$\sigma < 0.005$ mV	0	0
$\sigma < 0.010$ mV	0	0
$\sigma < 0.020$ mV	5	5
$\sigma < 0.050$ mV	4,119	3,416

Part 1: Reproduction of Published Results

3 Reproduction Methodology and Protocol

We shall adopt the pipeline introduced in the original paper to reproduce their results. In the upcoming subsections, we shall describe the methodologies and highlight the issues and/or limitations in their work. The results in this section are extracted from `reproduction.ipynb`.

3.0.1 Feature extraction

Prior to this stage, the **Data Preprocessing** stage flattens 12-lead ECG diagrams into a single vector, i.e. $(1000 \times 12) \rightarrow (12000,)$. We also noticed a critical typo issue in the original paper, stating that “ECG signal has 21000 dimensions”, but in fact, it only has 12000 dimensions, as written just before their section C in “III. METHODS”.

After that, we apply the `neurokit2` package [30] to extract the features, i.e. $(12000,) \rightarrow (16, N)$, where N is the number of peaks detected, and vary among ECG records. Moreover, as highlighted in [20], each lead in the 12-lead ECG are all comprising of the same peak amplitudes, intervals and segments as shown in **Fig. 2**, though each of which offers different viewpoints. However, as described, they flatten the 12 leads prior to extracting features, and thus, do not exactly conform to the medical meaning of these features. Therefore, it results in two hurdles they acknowledged later in their paper.

Besides, they do not present mathematical expressions to compute the intervals, hence, we shall rely on their description to work out these features, as below. $PQ = Q - P$, $ST = T - S$, $QT = T - Q$, $PR = R - P$, $QRS = S - Q$ and $RR = R_i - R_{i-1}$, where R, P, Q, S, T are five peak amplitudes.

The 16 features extracted by the following process. Firstly, we detect the R peak locations by using `ecg_peaks()`, subsequently utilise `ecg_delineate()` to obtain other peak locations using the R peaks as reference, and we set `method = "peak"` as [17] does not clearly specify, eventually calculate the intervals/segments by applying the expressions shown above. There exist null values during this process, and thus, we always take this issue into account at every step.

Table 2: 16 features extracted by `neurokit2` [17].

Feature group	Feature's name
Peak's positions	P_loc, Q_loc, R_loc, S_loc, T_loc
Peak's amplitude	P_amp, Q_amp, R_amp, S_amp, T_amp
Intervals and segments	PQ, ST, QT, PR, RR, QRS

3.0.2 Data preparation

As reported in the original paper, the difficulties do exist. First, there are 280 records (roughly 1.7%) whose 16 features contain missing values. Second, the number of R peaks across features differ significantly, i.e. 67 distinct feature lengths (min 0, median 12, max 81). Third, in 157 ECG records, the 16 features do not share the same number of R peaks. For brevity, we shall refer to R peaks as beats.

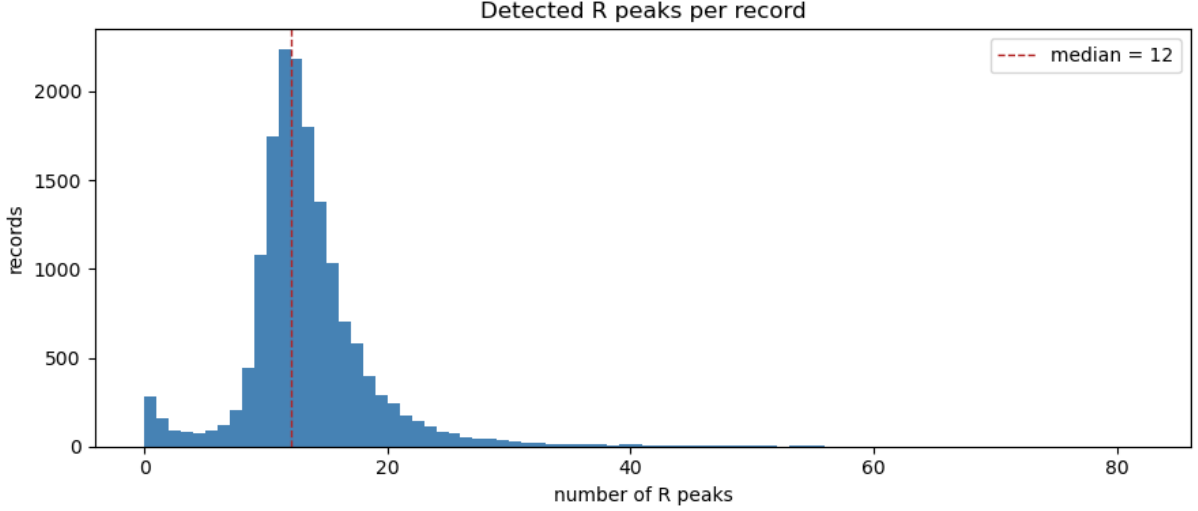


Figure 8: Number of R peaks across 16244 ECG records.

We shall resolve the above-mentioned issues as follow, strictly obey the order. First, we apply the Forward Filling (`ffill`) technique to impute the missing values presented in the dataset. Then, we equate the feature lengths across the dataset; however, [17] is unclear about this, we shall apply the median length, i.e. 12, which we believe it may balance between unnecessarily padding to compensate the length and dropping useful signals presented. Next, we flatten the features, i.e. $(16 \times 12) \rightarrow (192,)$.

We further realise that, an early application of `np.diff(R, prepend = np.nan)`, to compute RR, introduces some null values that `ffill` may again impute null values to the succeeding elements. Therefore, we complement the pipeline with another `ffill()`. Moreover, there exists a column with null values everywhere, and thus, we drop this column. Eventually, the number of features passed to the upcoming stages is 191. These subtle issues are not clearly explained in [17]. Then, we encode the diagnostic classes with numbers, in lieu of applying `LabelEncoder()` we strictly adhere to the following rules, established in [17].

Table 3: Label encoding of diagnostic classes [17].

Class	Numerical label
NORM	0
MI	1
STTC	2
HYP	3
CD	4

Next, we leverage the `train_test_split()` from `sklearn` to randomly partition the dataset into train/test sets using the 8/2 ratio. Nonetheless, the paper is unclear about what `random_state` they used and whether the attempted to partially resolve the imbalance issue by setting `stratify` during the split. We shall use `random_state = 42` and do not apply `stratify` to get the most imbalanced train/test sets, at least more imbalanced than applying `stratify`. Therefore, the imbalancing issue is somewhat preserved in both partitioned train and test sets, as demonstrated below.

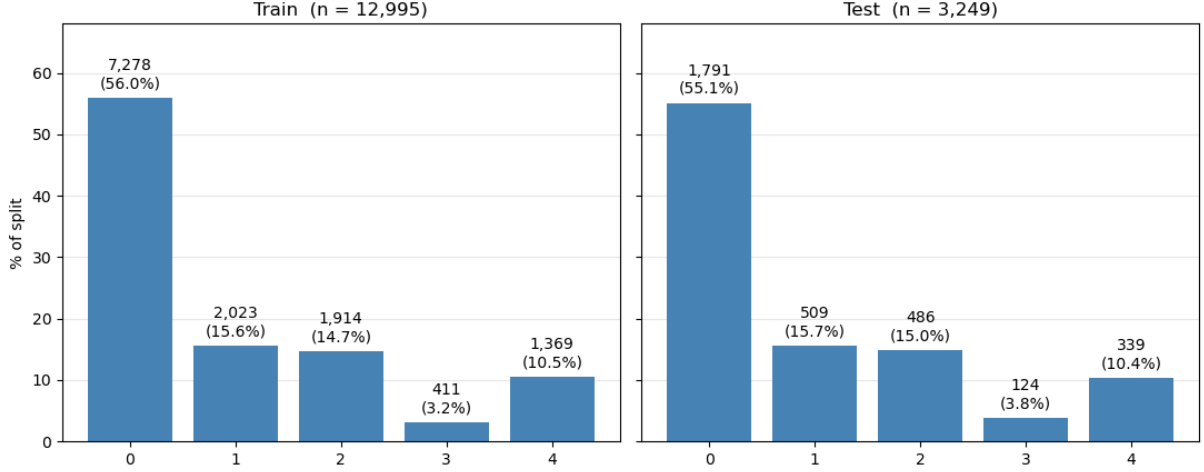


Figure 9: Class distribution of train and test sets

3.0.3 Data augmentation

It has been recognised and extensively studied in ML community that, performance of ML algorithms are adversely influenced by the presence of underrepresented data and/or skewness of class distributions [31]. Consequently, this class-imbalanced dataset amplifies the models’ biases towards predominant classes and may misclassify classes with fewer samples [17].

Having splitted the dataset, the original paper resolves class-imbalance issue by the utilisation of SMOTE, which stands for Synthetic Minority Over-sampling Technique [32]. A more thorough description of SMOTE can be found in [32]. In brief, given our dataset $\mathcal{D} \triangleq (\mathcal{X}, \mathcal{Y})$, consider the feature space $\mathcal{X}$, it randomly picks a sample, $\mathbf{x}_i \in \mathcal{X}$, from the minority class, $\mathbf{y}_i \in \mathcal{Y}$, then again, randomly selects one of its k nearest neighbours by KNN, says $\mathbf{x}_j \in \mathcal{X}$, and of couse, $\mathbf{y}_j = \mathbf{y}_i$. Next, it computes their difference, $\mathbf{d} = \mathbf{x}_i - \mathbf{x}_j$, and complementing the current dataset $\mathcal{D}$ by $\mathcal{X} \leftarrow \mathcal{X} \cup \{\mathbf{x}_i + \alpha \mathbf{d}\}$ and $\mathcal{Y} \leftarrow \mathcal{Y} \cup \{\mathbf{y}_i\}$, where $\alpha \sim U(0,1)$. Repeating this process untill all classes in $\mathcal{Y}$ share the same number of samples. We utilise `imblearn` as an alternative to manually re-implementing this technique. Also, we use the `random_state = 42`, since the original paper does not mention therein. Additionally, they also mention the feature scaling at the end of section “III.E”, though absence of the method, and thus, we shall apply `StandardScaler()` prior to SMOTE.

We further note that, the class-imbalance handling strategy is only applied to the train set, while the test set is left unchanged, otherwise, we would falsely report the actual performance of our models on the test set. Later on, we shall show that, following this approach cannot reproduce their results, whereas violating this approach, i.e. SMOTE before train/test split yields closer reproduced results.

Another unclear statement in their paper, they mentioned the process is applied to “80% of training data, which are selected randomly in each fold of the 10-fold cross-validation”. This is ambiguous, since in a standard 10-fold CV splits into 90% train and 10% test.

3.0.4 Modeling

We also noted another critical issue of the original paper, in their abstract, they stated the use of `AdaBoost`, also another typological error here, but later in the main body, they use `CatBoost`. Due to this inconsistency, we shall evaluate both `CatBoost` and `AdaBoost`.

In this experiment, we use, from `sklearn`: `KNeighborsClassifier`, `RandomForestClassifier`,

`AdaBoostClassifier` and leverage `HistGradientBoostingClassifier` as an alternative to `GradientBoostingClassifier` due to resources constraints, and they were empirically proved to have almost identical classification performance [33, 34, 35], from `catboost`: `CatBoostClassifier`, from `xgboost`: `XGBClassifier`. We omit the models’ descriptions, as they can be found in the corresponding papers and/or packages implementing them.

4 Reproduction Results and Critical Analysis

Having identified many inconsistencies and ambiguities in the original paper so far, we shall conduct the complete pipeline to reproduce their results. While working on this, we are aware of another critical inconsistency in their reported results, in some parts, they mention the use of 10-fold cross validation (CV), e.g. the tables presenting the per-class performance metrics of their models. Meanwhile, no explanation on the use of 10-fold CV nor single train/test split for the confusion matrices.

In **Sec. 3**, we introduce the methodologies and protocols in our pipeline as an attempt to reproduce the results presented in [17]. Nonetheless, they are quite fragmented, and thus, we shall unify them into one single pipeline as below.

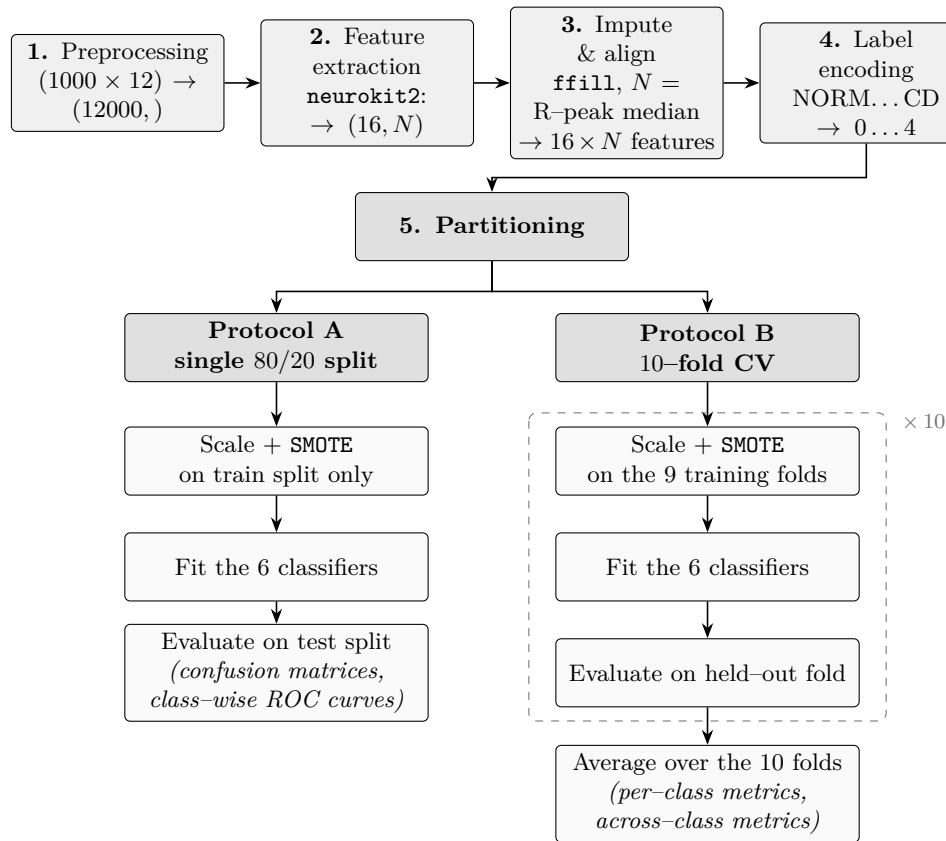


Figure 10: Unified reproduction pipeline.

4.1 Reproduced versus Published Results

We use `HistGradientBoostingClassifier` as alternative to `GradientBoostingClassifier` due to computing resources constraints, and their provably near-similar performance.

Table 4: **Protocol B** class-wise performance metrics across models.

(a) KNN				(b) Random Forest			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	64.7	23.7	34.7	NORM (0)	66.0	76.1	70.7
MI (1)	19.8	35.6	25.4	MI (1)	37.0	27.7	31.7
STTC (2)	18.5	29.3	22.7	STTC (2)	29.6	23.9	26.4
HYP (3)	16.7	34.9	22.5	HYP (3)	30.4	40.7	34.6
CD (4)	16.3	32.8	21.7	CD (4)	39.1	27.9	32.5

(c) Histogram-based Gradient Boosting				(d) AdaBoost			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	65.1	87.6	74.7	NORM (0)	67.4	39.8	49.9
MI (1)	45.9	26.1	33.2	MI (1)	21.6	27.2	23.9
STTC (2)	35.7	16.8	22.7	STTC (2)	22.0	29.8	25.2
HYP (3)	40.8	32.3	35.8	HYP (3)	14.8	62.9	23.9
CD (4)	48.0	28.9	36.0	CD (4)	20.3	25.1	22.3

(e) CatBoost				(f) XGBoost			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	66.0	85.0	74.3	NORM (0)	65.0	86.9	74.3
MI (1)	44.4	31.0	36.5	MI (1)	45.4	28.1	34.7
STTC (2)	35.1	19.0	24.5	STTC (2)	34.5	16.8	22.4
HYP (3)	35.9	29.7	32.1	HYP (3)	40.8	26.6	32.0
CD (4)	46.3	28.3	35.0	CD (4)	45.0	26.6	33.4

We present the table below to compare with their Table 12, and we complement it with a wall-clock time column, for sake of comparison against our proposed solution in [Part 2](#). More precisely, the time reported throughout this report refers to the time taken by the classifiers in **Protocol B**, i.e. 10-fold CV - including both training and testing, said differently, the actual inference speed is much shorter than that, and we omit the time taken by feature extractors.

Table 5: **Protocol B** Performance metrics across all models.

Metrics	XGBoost	RF	CatBoost	AdaBoost	GBoost	KNN
Precision	46.1	40.4	45.5	29.2	47.1	27.2
Recall	37.0	39.3	38.6	37.0	38.3	31.2
F1 Score	39.4	39.2	40.5	29.0	40.5	25.4
Accuracy	59.0	54.6	59.0	35.6	59.6	27.7
Time (s)	49.5	57.2	282.7	322.2	140.7	2.3

Figure 11: **Protocol A** Confusion matrices of all classifiers.

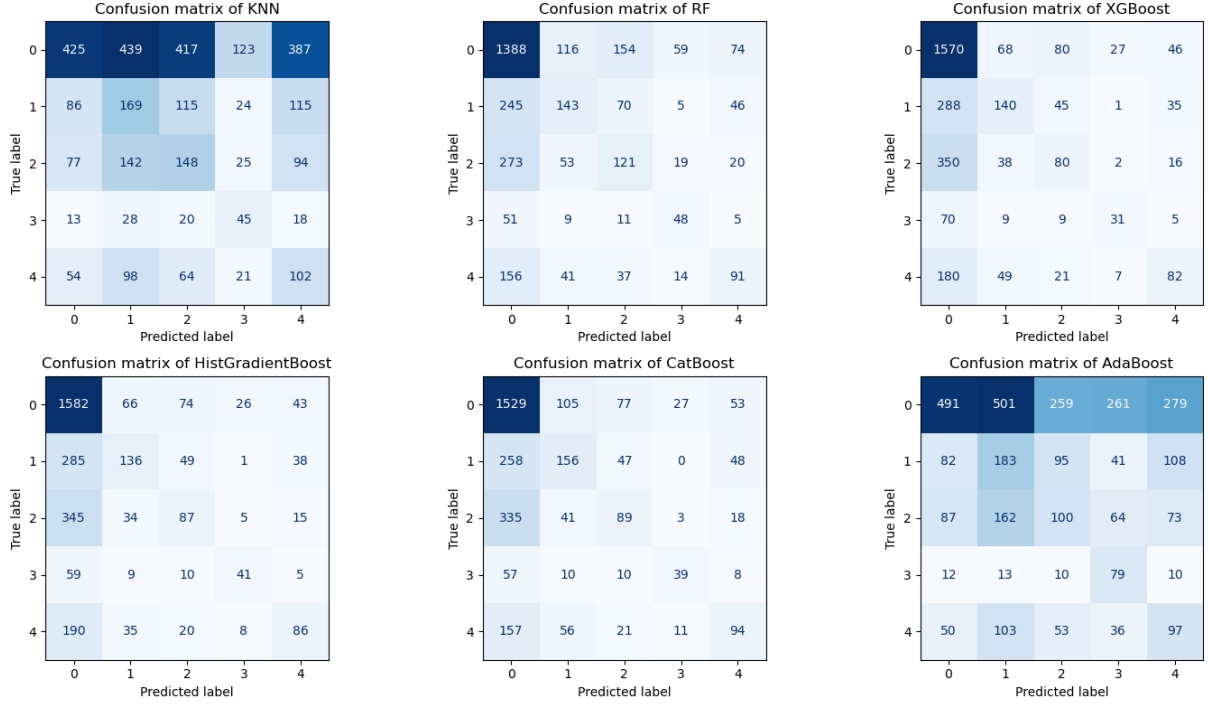
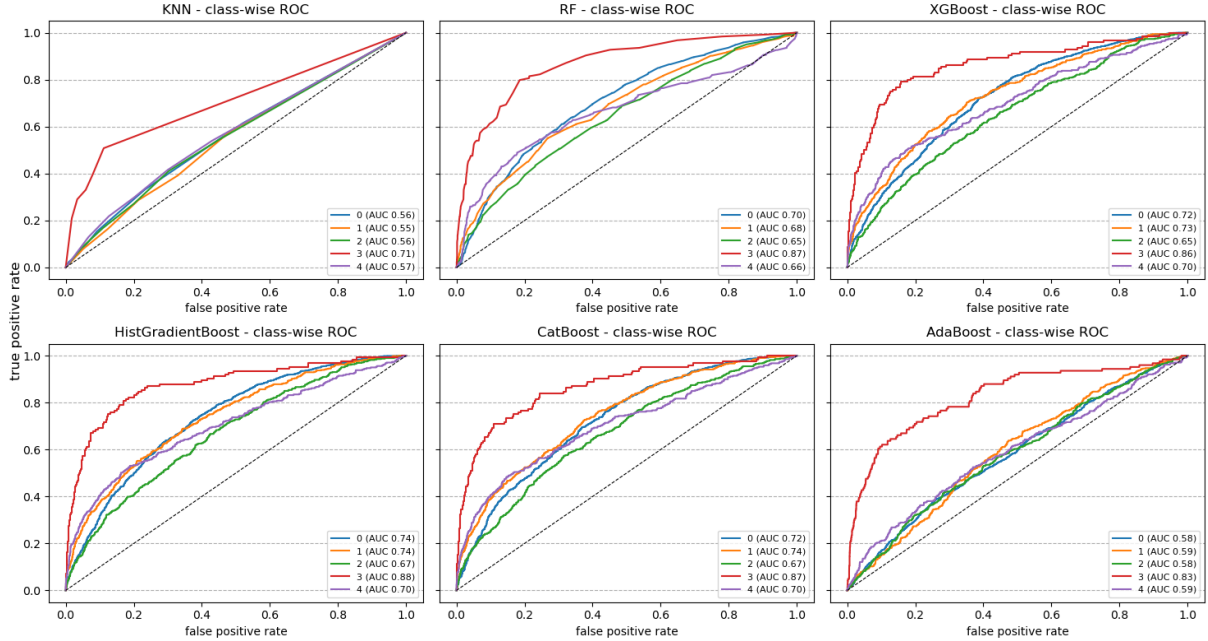


Figure 12: **Protocol A** ROC-AUC curves and scores across classifiers.



4.2 Critical Analysis of Discrepancies

From the outputs above, we are unable to reproduce the results presented in the original paper.

In particular, their Table 12 places XGBoost first at 93.0 F1 and 93.0% accuracy, followed by RF (92.0, 91.0%), CatBoost (89.0, 89.0%), G-Boost (87.0, 88.0%) and KNN (83.0, 82.0%). The corresponding entries of **Tab. 5** are 39.4, 39.2, 40.5, 40.5 and 25.4 F1, witnessing significant drops between 46 and 58. Noticeably, in their Tables 6, 7, 9, 10 and 11, HYP is the easiest

class for every model, at F1 96–99, whereas NORM is the hardest, at 56–86. In our **Tab. 4**, NORM is our strongest class throughout, at 34.7–74.7, and the three minority classes collapse to 21.7–36.0.

The **Fig. 11** is evaluated on held-out records, i.e. a single 80/20 train/test split. Therefore, the confusion matrices showcasing all models perform best for the most prevalent class in PTB-XL, i.e. NORM, and the models misclassify the other classes, mostly as the prevalent class. Meanwhile, their Figure 7, the distribution of classes are almost equivalent. More thorough quantitative observation is set out below.

Looking at the ROC curves for **XGBoost**, their Figure 8 reports class-wise AUC from 0.98 to 1.00, whereas our **Fig. 12** reports from 0.70 to 0.86, significantly diverge from their results. Although we agree with them that HYP is most easily separable class, their models, especially KNN captures all HYP classes in the test set is a clear signal of interpolation. In other words, the distribution of HYP in both train and test sets should be very similar, so that KNN can capture all of them, and thus, highly likely data leakage is presented, owing to SMOTE before train/test split.

Regarding the the extra model **AdaBoost**, our **CatBoost** attains 40.5 F1 at 59.0% accuracy against **AdaBoost**’s 29.0 and 35.6%, implying that, they indeed made a typo error in their abstract section.

We shall present some basic calculations based on their figures, and claim the inconsistencies throughout their paper herein.

Looking at their “Table 3. Distribution of diagnostic superclasses”, we have:

- SMOTE after train/test split
 - Total samples: $9528 + 4907 + 5250 + 2655 + 5486 = 27826$
 - 80% train: $27826 \times 80\% \approx 22261$
 - 20% test: $27826 \times 20\% \approx 5565$
- SMOTE before train/test split
 - Total samples: $9528 \times 5 = 47640$
 - 80% train: $47640 \times 80\% = 38112$
 - 20% test: $47640 \times 20\% = 9528$

Next, let us count the total samples in the test set, by taking into account their confusion matrices presented in their **Figure 7**. Since all matrices share the same number of test samples, consider “Confusion Matrix_KNN”: $414 + 547 + 544 + 118 + 231 + 28 + 1748 + 19 + 4 + 10 + 22 + 57 + 1694 + 4 + 10 + 1796 + 15 + 22 + 28 + 6 + 1766 = 9083$. As described in their section **III.E. Data Augmentation**, they stated “the process of data augmentation and feature scaling is done only on 80% of training data...”. We can exclude the possibility that 5565 is upsampled to 9083 test samples counted above. Therefore, it is more likely that they do SMOTE before train/test split. We also note here, another annotation errors, they name the subcaption (b) and (c) as “CM of RF”, while the subfigure (c) is “Confusion Matrix_XG”

Another apparent evidence is that, by taking the sum across the rows in the “Confusion Matrix_KNN”, we have:

- NORM (0): $414 + 547 + 544 + 118 + 231 = 1854$

- MI (1): $28 + 1748 + 19 + 4 + 10 = 1809$
- STTC (2): $22 + 57 + 1694 + 4 + 10 = 1787$
- HYP (3): $0 + 0 + 0 + 1796 + 0 = 1796$
- CD (4): $15 + 22 + 28 + 6 + 1766 = 1837$

The number of samples with respect to each class are almost identical, this is very unlikely to happen if they split the dataset prior to **SMOTE**, and thus, strongly supporting our point that they leak data into the training. Meanwhile, in their section **IV. Result and Discussion**, they stated “The Confusion Matrices for all the five classifiers with 20% of testing data are given in Figure 7.”, and indeed, the number of samples HYP (3), according to their Table 3, is 2655. There exists an extremely tiny probability for one to obtain 1796 test samples in HYP (3) for a 80/20 train/test split.

To reinforce our statements, we shall perform the following ablation study, conducting **SMOTE** before train/test split and some additional modifications to close the gap between their presented results and reproduced results. More importantly, the act of **SMOTE** prior to train/test split, is a kind of data leakage. We present the section below to argue their results, and in fact, the ablation studies below are the closest to their presented results.

4.3 Ablation study

We shall do the following, details can be found in `ablation.ipynb`. In brief, we perform `StandardScaler()` and **SMOTE** prior to the train/test split stage, as this would introduce data leakage to the train set, and in fact, significantly improve the results on the test set, though it falsely report models’ performance. We also realise that, either (1000×12) or (12×1000) can be flattened to $(12000,)$, the latter sounds more reasonable, as it places the lead axis before the time axis, and we hypothesize it would describe the raw ECG signals, more meaningfully and precisely. However, we still keep the same established flattening order, i.e. $(1000 \times 12) \rightarrow (12000,)$. Let us present the results obtained, by keeping the same configuration discussed earlier.

Figure 13: Comparison on class-wise precision for each model.

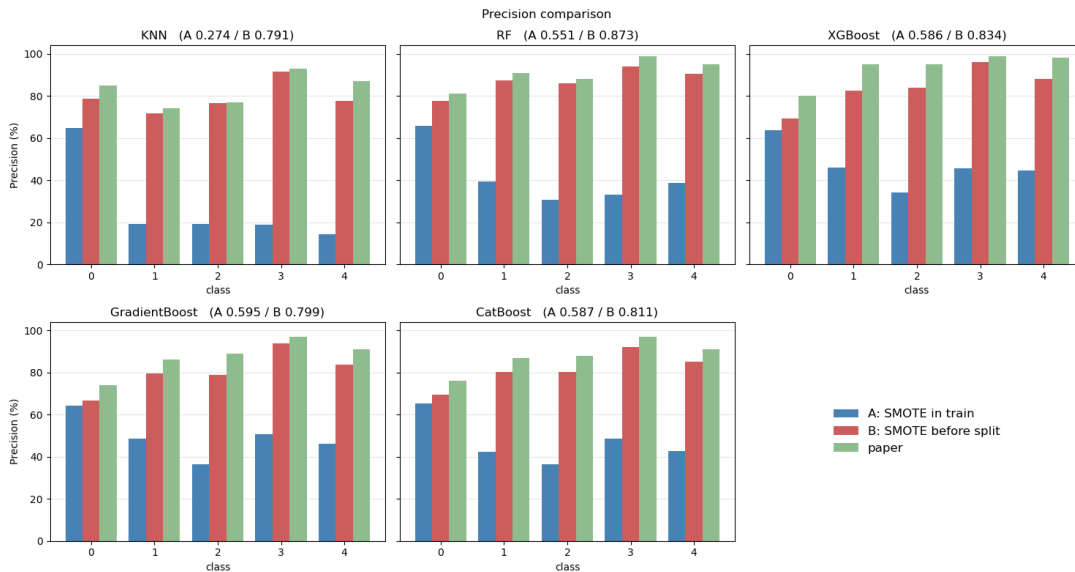


Figure 14: Comparison on class-wise recall for each model.

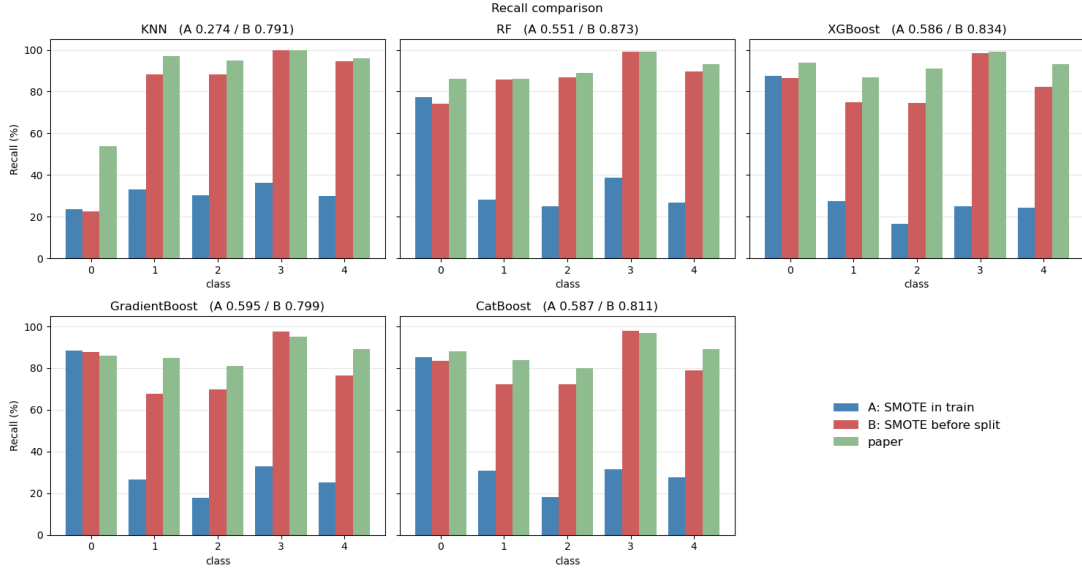
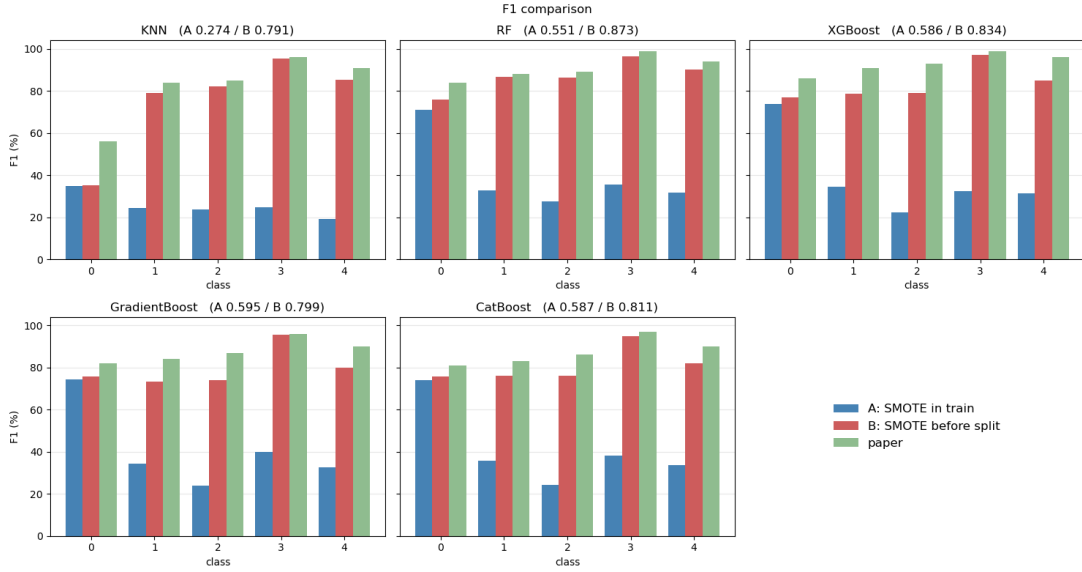


Figure 15: Comparison on class-wise F1 for each model.



The above-presented bar plots are to aid in the investigation of **Tab. 6** visually. Considering the figures and tables below, although there are moderate deviations from the actual results, we further notice the **Fig. 16** and **Fig. 17** share a very similar patterns. Interestingly, without further evidence, by changing the flattening strategy, $(12 \times 1000) \rightarrow (12000,)$, the gap is much narrower. One can test by switching “timemajor” to “leadmajor” in `ablation.ipynb`.

Table 6: **Protocol B** class-wise performance metrics across models.

(a) KNN				(b) Random Forest			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	81.8	23.0	35.9	NORM (0)	81.3	75.4	78.2
MI (1)	73.4	90.8	81.2	MI (1)	88.1	87.8	87.9
STTC (2)	77.3	90.6	83.4	STTC (2)	86.7	88.7	87.7
HYP (3)	92.4	99.9	96.0	HYP (3)	94.1	99.2	96.6
CD (4)	77.5	95.2	85.4	CD (4)	91.0	90.8	90.9

(c) Histogram-based Gradient Boosting				(d) AdaBoost			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	68.5	87.5	76.8	NORM (0)	43.4	38.6	40.6
MI (1)	79.6	69.8	74.4	MI (1)	32.5	32.2	32.0
STTC (2)	79.1	70.6	74.6	STTC (2)	31.5	32.3	31.6
HYP (3)	93.6	97.6	95.6	HYP (3)	58.7	71.2	64.3
CD (4)	84.5	77.0	80.5	CD (4)	37.5	32.1	34.4

(e) CatBoost				(f) XGBoost			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	70.4	84.6	76.9	NORM (0)	70.9	87.2	78.2
MI (1)	81.2	73.1	76.9	MI (1)	84.6	76.3	80.2
STTC (2)	80.3	73.3	76.6	STTC (2)	84.3	76.2	80.0
HYP (3)	92.5	98.0	95.2	HYP (3)	96.1	98.5	97.3
CD (4)	86.2	79.8	82.8	CD (4)	88.3	83.0	85.5

Table 7: **Protocol B** Performance metrics across all models.

Metrics	XGBoost	RF	CatBoost	AdaBoost	GBoost	KNN
Precision	84.8	88.2	82.1	40.7	81.1	80.5
Recall	84.2	88.4	81.8	41.3	80.5	79.9
F1 Score	84.3	88.3	81.7	40.6	80.4	76.4
Accuracy	84.2	88.4	81.8	41.3	80.5	79.9

Figure 16: **Protocol A** Confusion matrices of all classifiers.

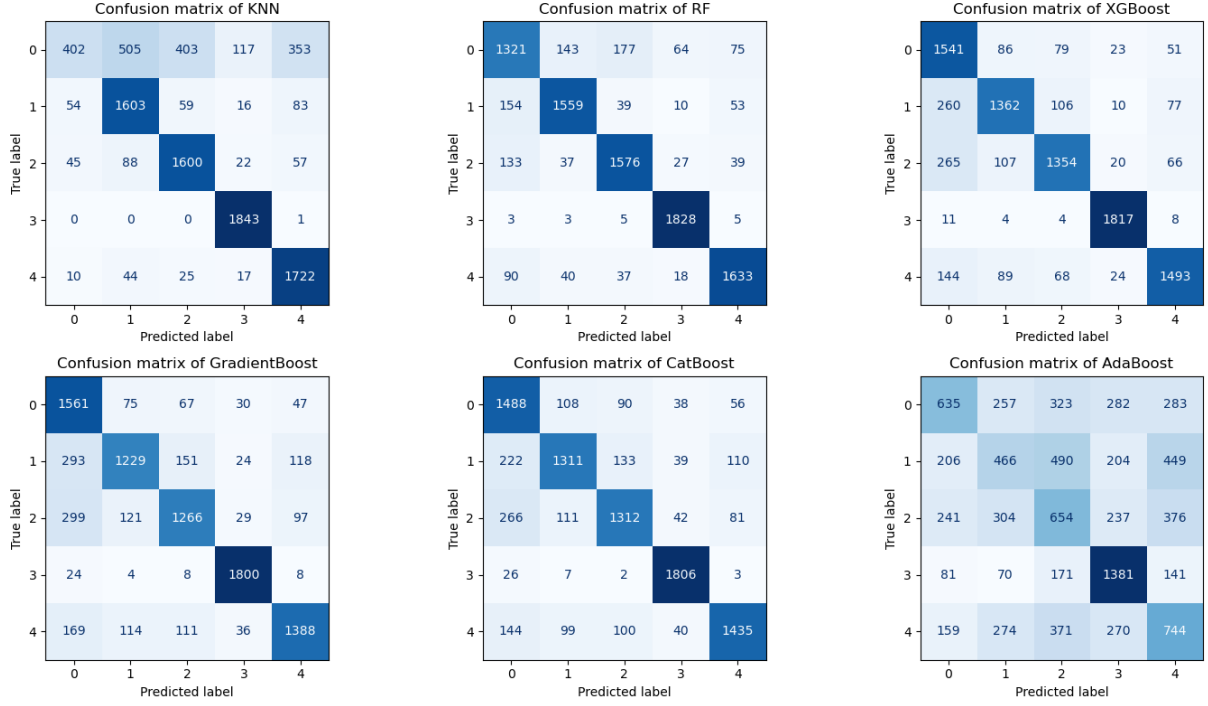
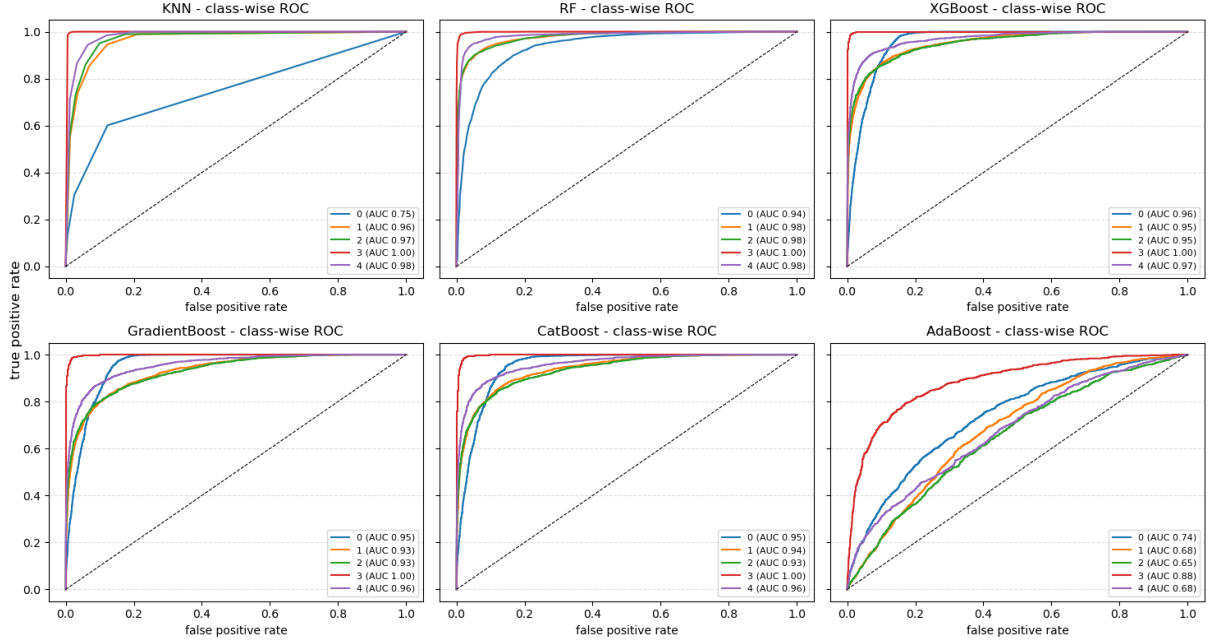


Figure 17: **Protocol A** ROC–AUC curves and scores across classifiers.



Findings. Performing scaling and SMOTE before partitioning takes the ensembles most of the way to the published level. As the flattening is unchanged from **Sec. 3**, the 39.9–49.1 F1 points the four ensembles gain over **Tab. 5** are attributable to this leakage alone; RF, for instance, rises from 39.2 to 88.3. The published figures are nevertheless not fully recovered. All 20 class-wise F1 scores of RF, XGBoost, GBoost and CatBoost in **Tab. 6** fall below the paper’s per-class

tables [17], by 0.1–13.0 points, the largest shortfall being STTC under **XGBoost** (80.0 against 93). **RF** comes closest, within 5.8 points on every class, and replaces **XGBoost** as the best model; the four ensembles span 80.5–88.4% accuracy, against 88.0–93.0% in their Table 12. For the boosted models, **NORM** precision (68.5–70.9) lies well below its recall (84.6–87.5), since **Fig. 16** shows minority records absorbed into **NORM**, e.g. 260 MI and 265 STTC records under **XGBoost**. **KNN** still fails on **NORM**, with F1 35.9 against the published 56, classifying only 402 of 1,780 **NORM** records correctly.

Three observations confirm that the leakage reproduces the paper’s protocol. First, recall equals accuracy for all six models in **Tab. 7**, which holds only when every class has equal support. Second, **Fig. 16** contains 9,069 test samples spread almost evenly across the classes (1,780–1,844 each), mirroring the 9,083 in the paper, whereas a leakage-free 80/20 split yields 3,249. Third, **HYP**, only 3.8% of a leakage-free test split, becomes the easiest class: its F1 is 95.2–97.3 for every model except **AdaBoost**, and its AUC in **Fig. 17** reaches 1.00 for those five models, matching the paper’s 1.00. **AdaBoost** remains far behind at 40.6 F1 and AUC 0.65–0.88.

Part 2: Proposed Machine Learning Solution

5 Motivation

In this section, to best compare to the previous results, we shall neither modify their choice of models nor SMOTE as an imbalance-handling strategy. Alternatively, we shall replace the feature extraction mechanism. Our proposed solution is based on the significance impact of feature engineering and extraction in Machine Learning community, which has been extensively studied [36, 37, 38, 39]. We shall devote this section to state our motivations of this work, as our attempt to achieve better results than the reproduced results in **Sec. 4**. Let us briefly discuss the general idea, more detailed descriptions are presented in **Sec. 6**.

Remarkably, as we identify the results presented in [17] is a misalignment with their experimental setup, and thus, we shall perform SMOTE after train/test split, to correctly report models' performance. Additionally, we disagree with their flattening approach, i.e. $\text{flatten}(12 \times 1000) \rightarrow (12000,)$, as this would break the temporal information and clinical meaning of ECG signals. Alternatively, we shall apply `neurokit2` to extract features from each ECG lead separately, then concatenate them, i.e. $(12 \times 1000) \rightarrow (12 \times 16) \rightarrow (192,)$.

In the realm of Deep Learning, 1D CNN and 2D CNN have been developed to better classify CVDs based on ECG signals [40, 41, 42]. Nonetheless, we shall avoid 2D CNN, as they require working with images, by converting ECG signals into spectrograms, which increase the computational complexity and memory usage of our models. We can then extract underlying features from raw ECG signals by applying 1D CNN on each ECG lead, passing through several temporal convolutional layers, and ultimately, we shall apply point-wise CNN kernel to fuse information across expanded leads to attain a $(125,)$ CNN-based feature vector.

By concatenating both hand-crafted `neurokit2` features and the CNN-based features, the resulting vectors become excessively long. In other words, the resulting vector belongs to an extremely high-dimensional space. An infamous problem associating with high-dimensional feature vector is curse of dimensionality, which amount to performance degradation of ML/DL models [43, 44, 45]. There are various dimensionality reduction techniques can be applied, as surveyed in [43, 45]. Driven by curiosity, we shall again utilise a DL-based dimensionality reduction technique, specifically training an Autoencoder [46], and subsequently utilise the encoder component to compress the aforementioned concatenated feature vector into a latent feature vector, and eventually feed it into the above-mentioned classifiers.

6 Proposed method

We offer **Fig. 18** to demonstrate our proposed solution, note the major modifications to **Fig. 10**. Noticeably, our architecture designs for Autoencoder and CNN were driven by our interest and curiosity, and thus, more advanced extensions are possible to improve classification outcomes.

First, we perform `neurokit2` on each ECG lead in a datum and concatenate across leads afterwards, while in their pipeline, they concatenate the ECG leads on a time major. As argued earlier, their strategy breaks the temporal information and clinical meaning of the ECG signals, as highlighted by [20], each ECG lead carries a different meaning, and the 16 features are extracted with respect to each ECG lead independently. Therefore, we conform to [20], then concatenate the `neurokit2`-based features. Doing this, we notice the number of missing values reduce substantially, which may benefit our models.

Second, we realise the limitation of hand-crafted features above, and some unexplainable yet subtle patterns may be exploited by DL models. Therefore, we utilise a temporal 1D convolutional neural network (1D CNN) to extract features directly from the raw ECG leads. Thereafter, we concatenate both hand-crafted features and CNN-based features as one high-dimensional feature vector.

Third, we understand the problem of sparsity and distance concentration in high-dimensional feature space [44]. Evidently, KNN suffers from it most severely, as observed in **Tab. 4**, **Tab. 5** and **Fig. 12**, owing to its direct dependence on distance metric. Meanwhile, tree-based models, though less directly depend on distance, are not totally immune to curse of dimensionality [47]. Therefore, we offer a dimensionality reducer f_θ , trained by Autoencoder framework [46], to compress into a lower-dimensional feature vector.

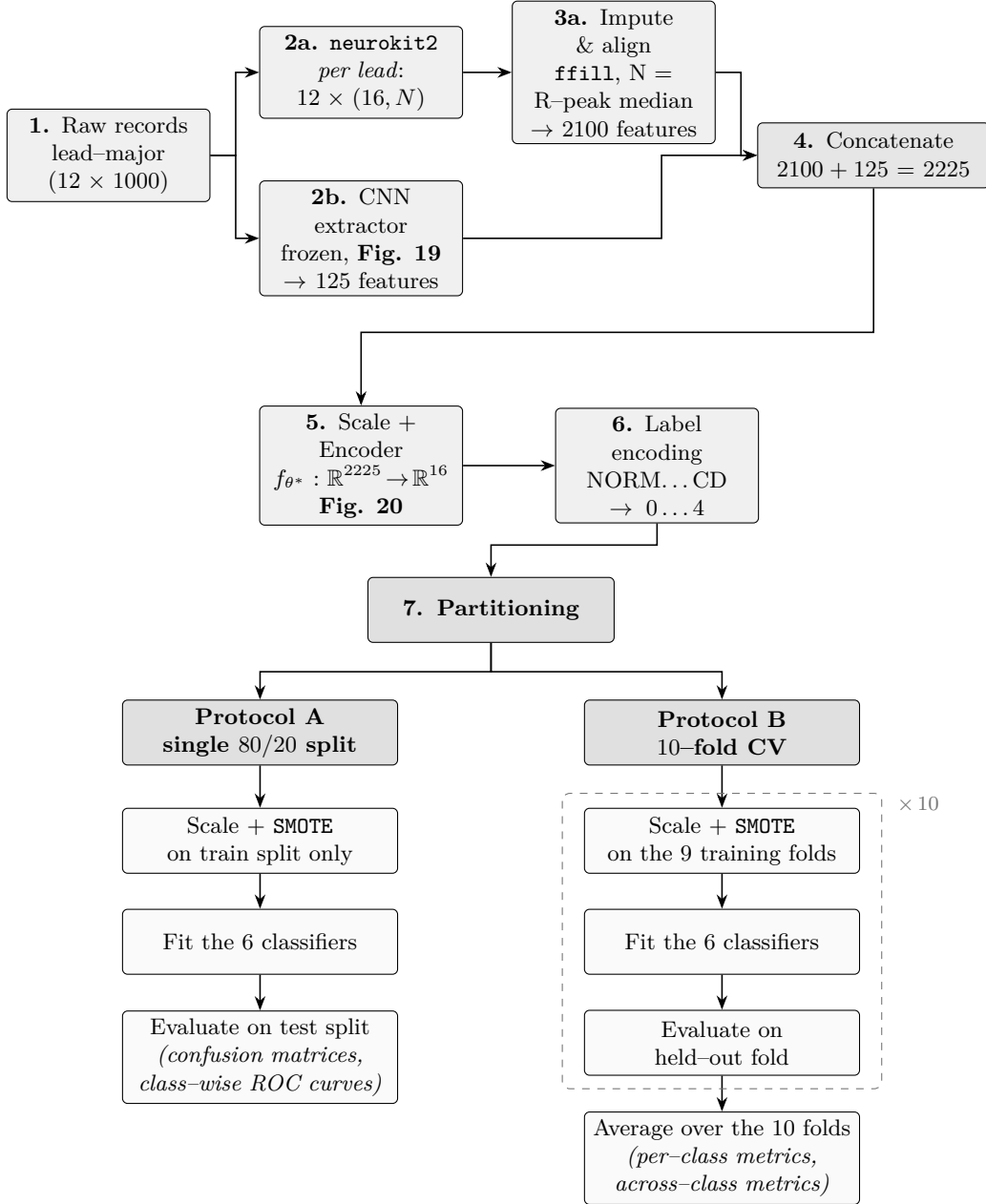


Figure 18: Our proposed pipeline. The two feature families (neurokit2 and 1D-CNN-based) are extracted from the same lead-major record, concatenated, and compressed by the frozen encoder before the data is partitioned.

We first detail the CNN branch of **Step 2b**, we further note that, the the CNN is only trained on the training data without touching the testing data, and thus, free of data leakage.

The model architecture below is inspired the idea of temporal CNN [48] though we exclude the pooling layer. At every layer, we add non-linearity by use of Rectified Linear Unit, shortly **ReLU**, and followed by 1D Batch Normalisation [49]. Besides, after each layer, we reduce the temporal dimension while expanding its channel depth (the ECG leads). Eventually, a cross-lead fusion is conducted by leveraging a point-wise kernel. To allow the model to learn from the data, we further apply Multi-layer Perceptrons (MLP) as a classifier and update the model by use of Cross-entropy loss, as we are working with multi-class classification problem. Remarkaly, the classification performance is attributed to both the CNN-based extractor and the discarded classifier head. Implementation of this model can be found in `CNN_extractor.py`.

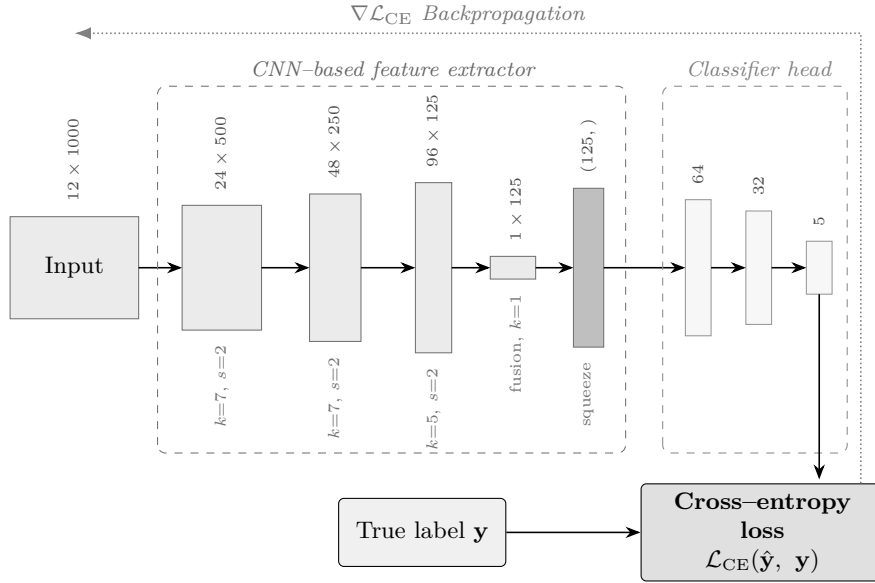


Figure 19: The 1D CNN feature extractor. Every convolution is followed by **BatchNorm1D** and **ReLU**. The pointwise 1×1 convolution collapses $96 \times 125 \rightarrow 1 \times 125$, and squeeze into 125-dimensional vector. The classifier head is discarded once training is done.

Next, we illustrate how the encoder f_{θ^*} of **Step 5** is obtained. The model architecture is simpler than the above-mentioned model. In particular, after each linear transformation layer, we add non-linearity by use of **ReLU**, and neither normalisation nor more advanced techniques are applied. The Encoder f_{θ} compresses the original feature into a so-called latent feature and the Decoder f_{ϕ}^{-1} reconstructs the original feature from the latent feature. We shall update f_{θ} and f_{ϕ}^{-1} by measuring the mean squared errors (MSE) between the original and the reconstructed vectors, a kind of self-supervised learning [50]. Doing this, f_{θ} is updated to reduce the dimensionality while maintaining the most information in the original feature, and f_{ϕ}^{-1} endeavours to reconstruct from the compressed feature. Considering the learning curves in **Fig. 21**, we shall claim our Autoencoder can compress and recover roughly 70% variance in the concatenated feature vector. Implementation can be found in `Autoencoder.py`.

We do not use denoising autoencoder [46], as we cannot distinguish between clean and noisy data providing the current PTB-XL [28]. Said differently, we are unsure whether the given dataset only contains clean signals or not, and thus, adding noise to the training data may introduce more errors rather than robustness improvement for Autoencoder.

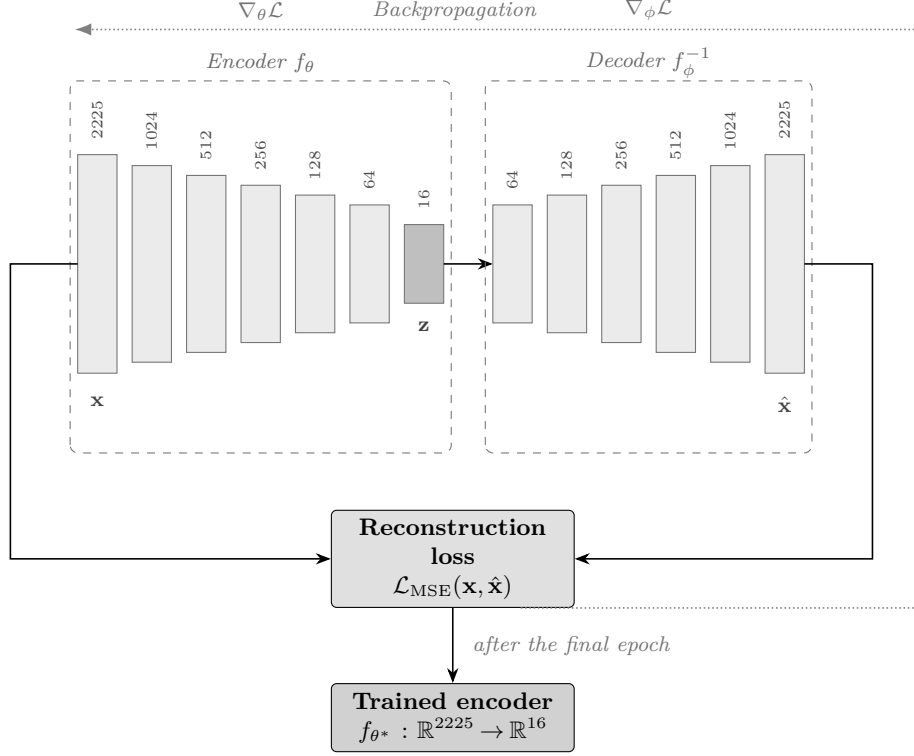


Figure 20: Autoencoder training procedure used to learn the encoder f_{θ^*} . We apply **ReLU** after each layer, except the final layer in f_θ and f_ϕ^{-1} . The decoder is discarded once training is done.

We offer the following figures to demonstrate the aforementioned models are capable of learning from empirical data and progressively updating their knowledge. Note that, as we do not evaluate validation accuracy, we shall use the learned parameters in the final epoch as our model in **Fig. 18**. Training procedures were done on High-Performance Computing (HPC) clusters, details can be found in `train.py` and `bash.sh`. The hardware configuration is AMD EPYC 7402P (24 cores - 48 threads), 256 GB DDR4 RAM and 4x NVIDIA A100 (40GB HBM2 each).

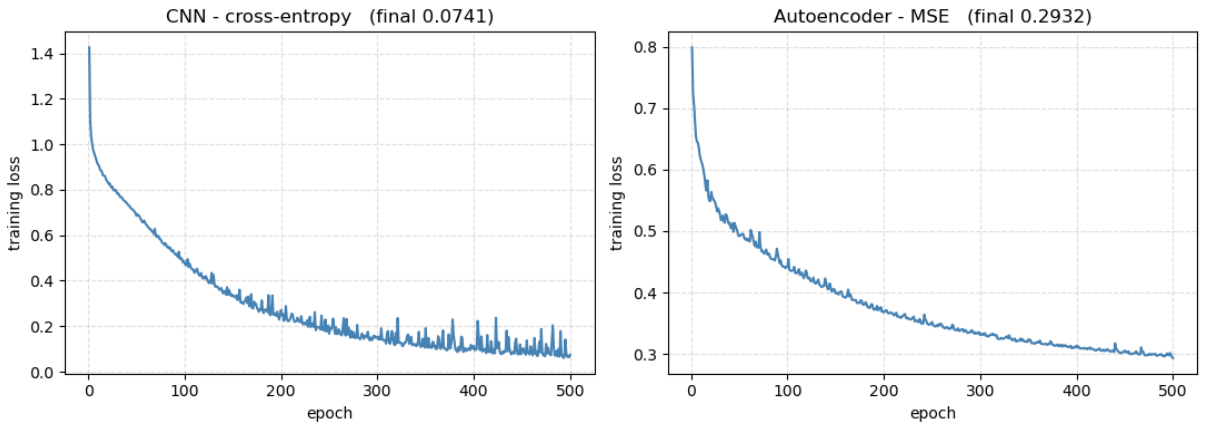


Figure 21: Learning curves of 1D CNN feature extractor (**Fig. 19**) and Autoencoder (**Fig. 20**).

7 Results and Comparative Analysis

We devote this section to deliver the classification performance of our proposed pipeline. Details can be found in `my_notebook.ipynb`. As described in **Sec. 6**, **Protocol A** is free of data

leakage, but **Protocol B** is not totally free of it, and varies differently depending on each fold. This does not affect our ablation studies in **Sec. 4.3**, because the data leakage is attributed to our newly introduced CNN-based feature extractors and feature compressor.

Table 8: **Protocol B** class-wise performance metrics across models.

(a) KNN				(b) Random Forest			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	81.8	61.5	70.2	NORM (0)	82.6	80.8	81.7
MI (1)	39.0	43.9	41.3	MI (1)	51.4	50.8	51.1
STTC (2)	40.3	49.1	44.2	STTC (2)	51.8	51.4	51.5
HYP (3)	18.4	50.4	26.9	HYP (3)	30.3	38.4	33.7
CD (4)	50.9	64.9	56.9	CD (4)	64.3	68.2	66.1

(c) Histogram-based Gradient Boosting				(d) AdaBoost			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	87.2	73.1	79.5	NORM (0)	82.7	64.1	72.2
MI (1)	48.2	53.8	50.8	MI (1)	37.4	39.7	38.4
STTC (2)	50.8	59.6	54.8	STTC (2)	37.2	41.8	39.1
HYP (3)	26.6	51.9	35.1	HYP (3)	14.1	47.2	21.7
CD (4)	61.5	70.2	65.5	CD (4)	46.4	54.0	49.6

(e) CatBoost				(f) XGBoost			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	86.5	75.0	80.3	NORM (0)	85.6	74.7	79.7
MI (1)	49.6	54.3	51.8	MI (1)	48.4	53.4	50.8
STTC (2)	52.2	60.2	55.9	STTC (2)	52.2	60.0	55.7
HYP (3)	28.9	52.8	37.3	HYP (3)	28.4	46.6	35.2
CD (4)	64.1	69.9	66.8	CD (4)	62.5	69.6	65.8

We remark that, **Tab. 9** and **Tab. 8** do not totally correctly reflect models’ performance on 10-fold CV. As we mentioned earlier, the extractors were trained on a 80% train data. Meanwhile, in **Protocol B**, we split in 10 different folds to evaluate the models’ performance. Therefore, in each 10% test of each fold, there may contain some leak training data. On the other hand, this also implies there are some unseen samples in the train set. Together, we believe this may alleviate the impact of data leakage on **Protocol B**. However, as an excuse, owing to resources constraints, we are unable to re-train CNN-based extractor and encoder on all 10 folds.

Table 9: **Protocol B** Performance metrics across all models.

Metrics	XGBoost	RF	CatBoost	AdaBoost	GBoost	KNN
Precision	55.4	56.1	56.3	43.6	54.8	46.1
Recall	60.8	57.9	62.5	49.3	61.7	53.9
F1 Score	57.5	56.8	58.4	44.2	57.1	47.9
Accuracy	67.7	69.1	68.4	55.4	67.1	56.9
Time (s)	9.2	17.4	57.2	33.0	52.8	0.9

Considering the tables above, our pipeline improves every classifier over the reproduction in **Tab. 5**. Macro F1 rises by 16.6–18.1 points for the four ensembles and by 22.5 for KNN (25.4 $\rightarrow$ 47.9), the largest gain of all. The ensembles lie within 1.6 F1 of one another (56.8–58.4): **CatBoost** attains the highest F1 and **RF** the highest accuracy (69.1%). Fitting the classifiers is also 2.6–9.8 $\times$ faster, e.g. 322.2 $\rightarrow$ 33.0 s for **AdaBoost**, although this excludes the feature extractors.

Class-wise (**Tab. 8** against **Tab. 4**), the largest average F1 gains are on CD (+31.6) and STTC (+26.2). HYP barely moves: its F1 stays at 21.7–37.3 and falls for three of the six models, while its precision (14.1–30.3) is far below its recall (38.4–52.8), i.e. every model over-predicts the rarest class.

The held-out split suggests the leakage in **Protocol B** is modest. On the 3,249 unseen records of **Fig. 22**, the ensembles reach 64.9–66.8% accuracy, 1.0–2.3 points below their 10-fold scores, whereas in **Part 1** this gap stayed within 0.5 for the same models. In **Fig. 23**, their class-wise AUCs rise to 0.81–0.90 from 0.65–0.88 in **Fig. 12**, with MI the hardest class (0.81–0.83) and CD the easiest (0.89–0.90).

Figure 22: **Protocol A** Confusion matrices of all classifiers.

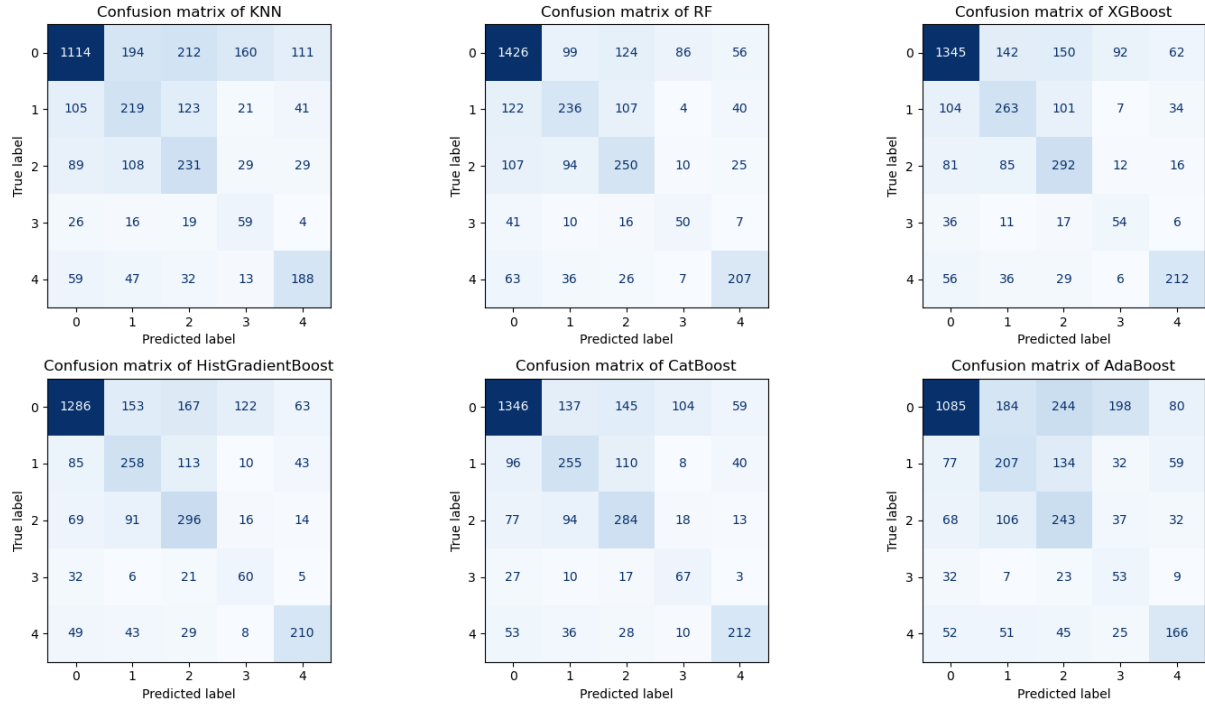
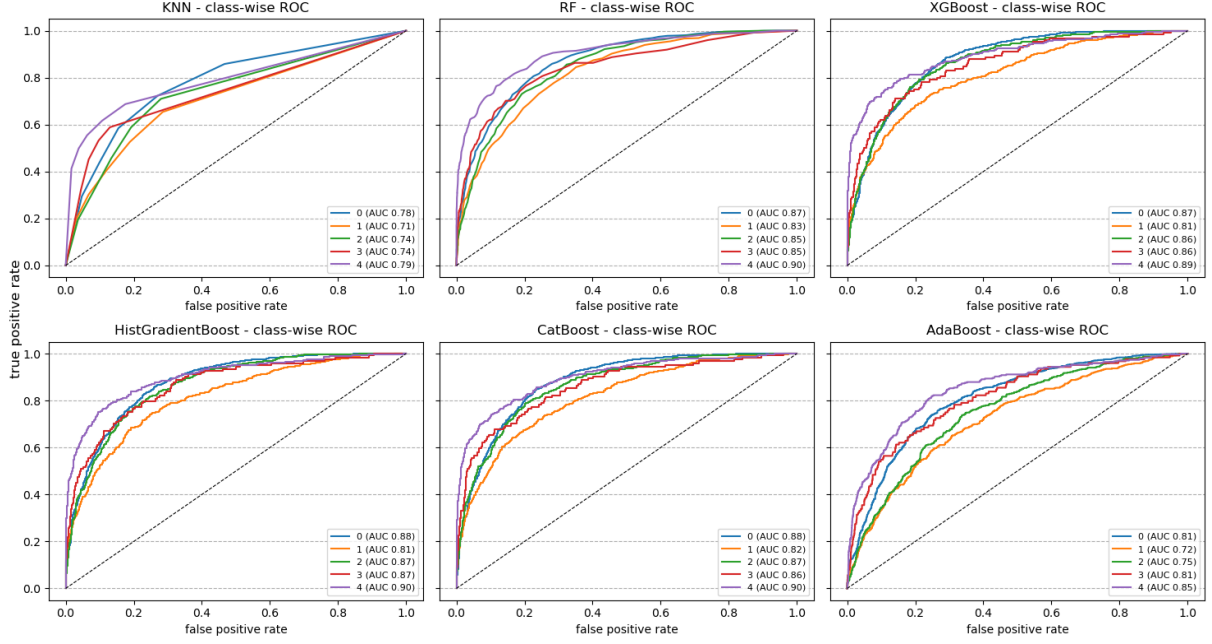


Figure 23: **Protocol A** ROC–AUC curves and scores across classifiers.



8 Ablation study: Full vs. Compressed feature vectors

A problem continuously mentioned throughout this report is Curse of Dimensionality. Undoubtedly, KNN, a distance-based ML method, suffers from it. Nonetheless, tree-based models are less prone to this problem, on-going debates over tree models performance on high-dimensional data, though mostly favour feasibility of tree models when coping with high dimensionality [47, 51, 52, 53]. We shall empirically challenge two viewpoints, by keeping and excluding the encoder, and later realise, either of which offer their own pros and cons. We present the results as below. By the same caution raised in **Sec. 7, Protocol B** is slightly misleading due to data leakage.

We offer an additional table recording the time taken by classifiers in both training and testing.

Table 10: **Protocol A** Held-out fitting and testing time (s).

Compressed?	XGBoost	RF	CatBoost	AdaBoost	GBoost	KNN
No	58.8	20.7	306.0	312.1	59.7	6.1
Yes	0.8	1.5	6.1	2.9	5.3	0.2
Improved?	73.5x	13.8x	50.2x	107.6x	11.3x	30.5x

Clearly, the total time taken to train and test the aforementioned classifiers substantially differ. We further note that, for a trained 1D-CNN-based extractor and **encoder**, the inference speed is not a bottleneck.

Table 11: **Protocol B** class-wise performance metrics across models.

(a) KNN				(b) Random Forest			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	89.4	24.0	37.8	NORM (0)	83.5	84.6	84.1
MI (1)	30.7	52.8	38.8	MI (1)	65.6	59.9	62.6
STTC (2)	32.1	64.0	42.8	STTC (2)	63.0	65.0	63.9
HYP (3)	15.7	57.7	24.5	HYP (3)	43.4	50.1	46.3
CD (4)	41.4	65.4	50.6	CD (4)	73.0	70.7	71.8

(c) Histogram-based Gradient Boosting				(d) AdaBoost			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	88.5	93.5	90.9	NORM (0)	84.6	66.0	74.1
MI (1)	77.7	73.2	75.4	MI (1)	43.5	48.5	45.7
STTC (2)	77.4	74.4	75.8	STTC (2)	48.9	62.9	54.9
HYP (3)	64.8	49.9	56.0	HYP (3)	24.9	59.1	34.8
CD (4)	84.8	77.7	81.0	CD (4)	51.8	59.5	55.0

(e) CatBoost				(f) XGBoost			
Class	Precision	Recall	F1 Score	Class	Precision	Recall	F1 Score
NORM (0)	89.8	93.2	91.5	NORM (0)	87.4	93.8	90.5
MI (1)	78.8	74.4	76.5	MI (1)	78.3	71.6	74.7
STTC (2)	76.8	76.9	76.8	STTC (2)	76.6	73.8	75.1
HYP (3)	63.6	53.8	57.8	HYP (3)	63.6	45.4	52.8
CD (4)	82.6	77.2	79.8	CD (4)	84.6	74.6	79.2

Table 12: **Protocol B** Performance metrics across all models.

Metrics	XGBoost	RF	CatBoost	AdaBoost	GBoost	KNN
Precision	78.1	65.7	78.3	50.7	78.6	41.9
Recall	71.8	66.0	75.1	59.2	73.7	52.8
F1 Score	74.5	65.7	76.5	52.9	75.8	38.9
Accuracy	83.8	75.3	84.9	61.9	84.4	39.9
Time (s)	647.0	246.1	2903.3	3493.7	646.5	28.2

Considering the tables above, removing the compression stage divides the classifiers. Given all 2,225 features, the boosted ensembles gain 17.0–18.7 F1 points over **Tab. 9**, **CatBoost** reaching 76.5, while **RF** and **AdaBoost** gain 8.9 and 8.7. **KNN** is the only model to deteriorate, from 47.9 to 38.9 F1, notably for **NORM**, its recall collapses to 24.0, only 416 of 1,791 **NORM** records are recognised in **Fig. 24**. Tree-based models thus exploit the high-dimensional input, whereas the distance-based **KNN** depends on compression. The gain is costly, as fitting takes 12–106× longer, e.g. $57.2 \rightarrow 2,903.3$ s for **CatBoost**.

Looking at the single 80/20 held-out evaluation. In **Fig. 24**, **XGBoost**, **GBoost** and **CatBoost** reach only 74.9–75.0% accuracy, against 1.0–2.1 for the remaining models here and 1.0–2.3 in **Sec. 7**. Even so, held-out accuracy still improves over the compressed pipeline by 8.3–10.1

points for these three models and 7.5 for RF, while KNN loses 17.1. Class-wise AUCs in **Fig. 25** rise to 0.87–0.93 for the ensembles, from 0.81–0.90, and HYP becomes the lowest-scoring class for XGBoost, GBoost and CatBoost.

Figure 24: **Protocol A** Confusion matrices of all classifiers.

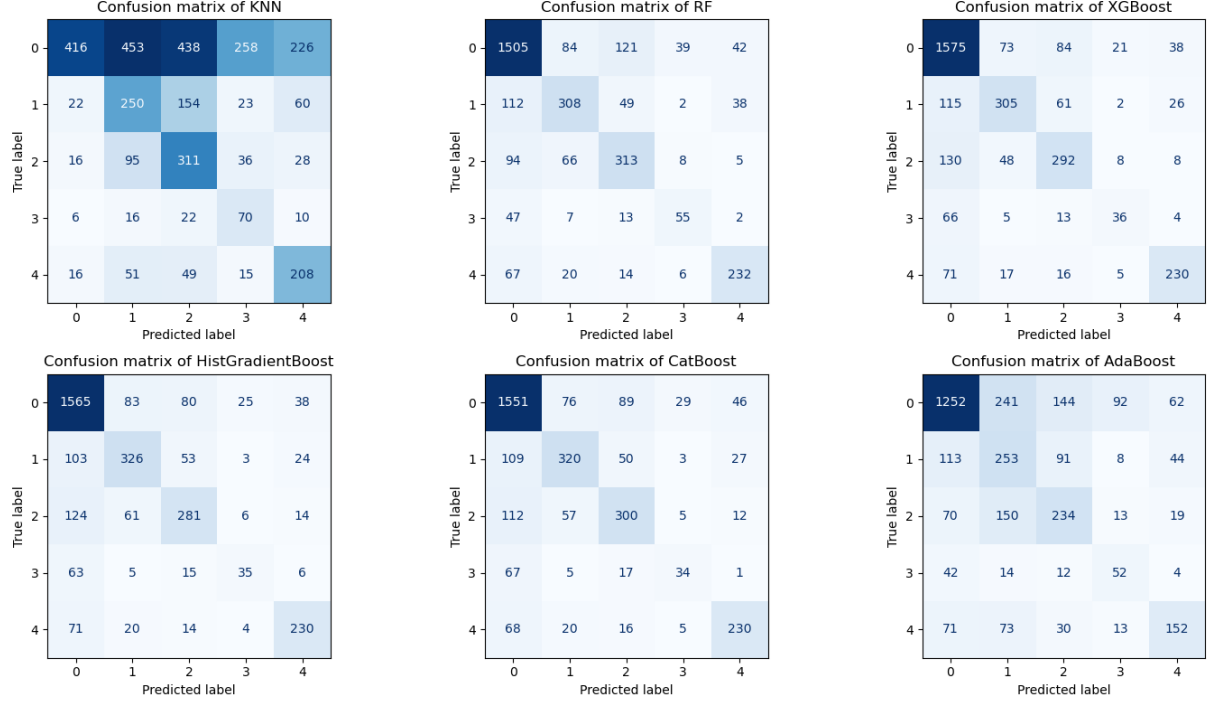
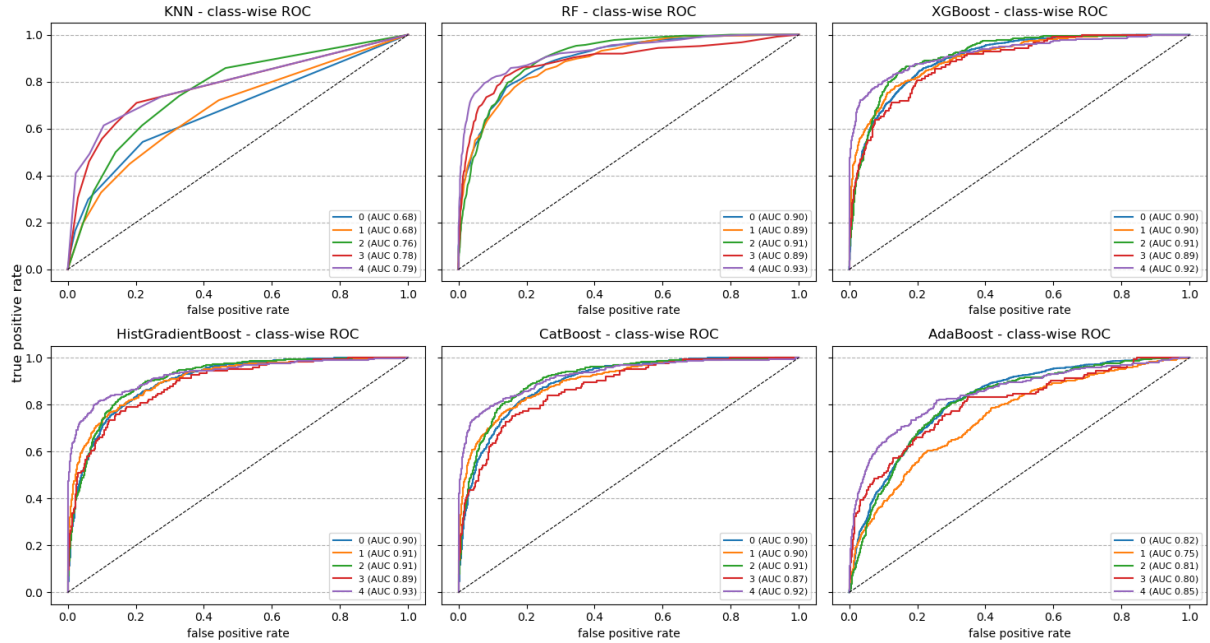


Figure 25: **Protocol A** ROC–AUC curves and scores across classifiers.



9 Conclusion and Discussion

In this report, we failed to reproduce the results presented in [17]; however, we found evidence in their paper that led to their SoTA results. Additionally, we identify critical errors throughout their paper, as raised in **Sec. 4.2**. Therefore, though beyond the purpose of this study, we call for an in-depth revision of this paper [17]. From the evidence above, e.g. a balanced test set of 9,083 records, we believe they applied **SMOTE** before the train/test split, introducing data leakage to their test set, and thus, ideal results were obtained. To verify this conjecture, we also conduct an ablation study by leaking data in the same way, and obtain close, though not identical, results, i.e. 80.5–88.4% accuracy for the ensembles against their 88.0–93.0%.

Having criticised their paper, we shall propose our Machine Learning solution to the same task, though we compare it to the reproduced results, not their presented results. In our solution, we leverage a 1D CNN to extract hidden features that may not be captured by the hand-crafted features extracted by **neurokit2**. We then train an Autoencoder, and utilise its Encoder to compress the 2,225 concatenated features into a 16-dimensional latent vector, which is fed to the same set of classifiers used in [17]. Empirically, the results are only above-average, the ensembles reaching 56.8–58.4 F1, up from 39.2–40.5 in the reproduction, and thus, require more enhancements to reach product quality. Remarkably, we also conduct an ablation study to compare whether compressed or uncompressed features benefit the selected classifiers the most. Interestingly, the tree-based models gain 8.7–18.7 F1 points by directly training and testing on uncompressed data, though fitting takes 12–106× longer. Meanwhile, due to the Curse of Dimensionality, KNN (a distance-based classifier) performs poorly on uncompressed data, dropping from 47.9 to 38.9 F1.

References

- [1] World Health Organization, “Cardiovascular diseases (CVDs).” WHO Fact Sheet, July 2025. Updated 31 July 2025. Available: [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)).
- [2] M. Lindstrom, N. DeCleene, H. Dorsey, V. Fuster, C. O. Johnson, K. E. LeGrand, G. A. Mensah, C. Razo, B. Stark, J. Varieur Turco, and G. A. Roth, “Global burden of cardiovascular diseases and risks collaboration, 1990-2021,” *Journal of the American College of Cardiology*, vol. 80, no. 25, pp. 2372–2425, 2022.
- [3] S. S. Martin, A. W. Aday, N. B. Allen, Z. I. Almarzooq, C. A. M. Anderson, P. Arora, C. L. Avery, C. M. Baker-Smith, N. Bansal, A. Z. Beaton, Y. Commodore-Mensah, M. E. Currie, M. S. V. Elkind, W. Fan, G. Generoso, B. B. Gibbs, D. G. Heard, S. Hiremath, M. C. Johansen, D. S. Kazi, A. H. A. C. on Epidemiology, P. S. Committee, and S. S. Committee, “2025 heart disease and stroke statistics: A report of us and global data from the american heart association,” *Circulation*, vol. 151, no. 8, pp. e41–e660, 2025.
- [4] World Heart Federation, “World heart report 2023: Confronting the world’s number one killer,” tech. rep., World Heart Federation, Geneva, Switzerland, 2023. Available: <https://world-heart-federation.org/wp-content/uploads/World-Heart-Report-2023.pdf>.
- [5] World Health Organization, *World Health Statistics 2025: Monitoring Health for the SDGs, Sustainable Development Goals*. World Health Organization, 2025.
- [6] S. B. Shuvo, S. N. Ali, S. I. Swapnil, M. S. Al-Rakhami, and A. Gumaei, “CardioXNet: A novel lightweight deep learning framework for cardiovascular disease classification using heart sound recordings,” *IEEE Access*, vol. 9, pp. 36955–36967, 2021.
- [7] M. A. Simonyan, Y. M. Ishbulatov, O. M. Posnenkova, V. A. Shvartz, A. S. Karavaev, V. V. Skazkina, V. I. Gridnev, R. V. Ukolov, and A. R. Kiselev, “Spectral analysis of photoplethysmography signal in patients with cardiovascular diseases and healthy subjects,” in *Proc. 5th Scientific School on Dynamics of Complex Networks and their Applications (DCNA)*, pp. 180–182, 2021.
- [8] A. J. Rogers, J. M. Miller, R. Kannappan, and P. Sethu, “Cardiac tissue chips (ctcs) for modeling cardiovascular disease,” *IEEE Transactions on Biomedical Engineering*, vol. 66, no. 12, pp. 3436–3443, 2019.
- [9] A. Nikam, S. Bhandari, A. Mhaske, and S. Mantri, “Cardiovascular disease prediction using machine learning models,” in *Proc. IEEE Pune Section Int. Conf. (PuneCon)*, pp. 22–27, 2020.
- [10] X. Qian, Y. Li, X. Zhang, H. Guo, J. He, X. Wang, Y. Yan, J. Ma, R. Ma, and S. Guo, “A cardiovascular disease prediction model based on routine physical examination indicators using machine learning methods: A cohort study,” *Frontiers in Cardiovascular Medicine*, vol. 9, p. 854287, 2022.
- [11] A. Rahim, Y. Rasheed, F. Azam, M. W. Anwar, M. A. Rahim, and A. W. Muzaffar, “An integrated machine learning framework for effective prediction of cardiovascular diseases,” *IEEE Access*, vol. 9, pp. 106575–106588, 2021.
- [12] H. Yang, Z. Chen, H. Yang, and M. Tian, “Predicting coronary heart disease using an improved LightGBM model: Performance analysis and comparison,” *IEEE Access*, vol. 11, pp. 23366–23380, 2023.

- [13] B. Akkaya, E. Sener, and C. Gursu, “A comparative study of heart disease prediction using machine learning techniques,” in *Proc. Int. Congr. Human-Computer Interaction, Optimization and Robotic Applications (HORA)*, pp. 1–8, 2022.
- [14] D. Chicco, C. A. Lovejoy, and L. Oneto, “A machine learning analysis of health records of patients with chronic kidney disease at risk of cardiovascular disease,” *IEEE Access*, vol. 9, pp. 165132–165144, 2021.
- [15] S. Ghorashi, K. Rehman, A. Riaz, H. K. Alkahtani, A. H. Samak, I. Cherrez-Ojeda, and A. Parveen, “Leveraging regression analysis to predict overlapping symptoms of cardiovascular diseases,” *IEEE Access*, vol. 11, pp. 60254–60266, 2023.
- [16] G. Joo, Y. Song, H. Im, and J. Park, “Clinical implication of machine learning in predicting the occurrence of cardiovascular disease using big data (nationwide cohort data in korea),” *IEEE Access*, vol. 8, pp. 157643–157653, 2020.
- [17] N. Sinha, M. A. G. Kumar, A. M. Joshi, and L. R. Cenkeramaddi, “DASMcC: Data augmented SMOTE multi-class classifier for prediction of cardiovascular diseases using time series features,” *IEEE Access*, vol. 11, pp. 117643–117656, 2023.
- [18] K. C. Siontis, P. A. Noseworthy, Z. I. Attia, and P. A. Friedman, “Artificial intelligence-enhanced electrocardiography in cardiovascular disease management,” *Nature Reviews Cardiology*, vol. 18, no. 7, pp. 465–478, 2021.
- [19] N. Strodthoff, P. Wagner, T. Schaeffter, and W. Samek, “Deep learning for ECG analysis: Benchmarks and insights from PTB-XL,” *IEEE Journal of Biomedical and Health Informatics*, vol. 25, no. 5, pp. 1519–1528, 2021.
- [20] A. L. Goldberger, “Introductory remarks,” in *Goldberger’s Clinical Electrocardiography (Ninth Edition)*, pp. xi–xii, Elsevier, ninth edition ed., 2018.
- [21] M. P. Witvliet, E. P. F. Karregat, J. C. L. Himmelreich, J. S. S. G. de Jong, W. A. M. Lucassen, and R. E. Harskamp, “Usefulness, pitfalls and interpretation of handheld single-lead electrocardiograms,” *Journal of Electrocardiology*, vol. 66, pp. 33–37, 2021.
- [22] K. Rajakariar, A. N. Koshy, J. K. Sajeev, S. Nair, L. Roberts, and A. W. Teh, “Accuracy of a smartwatch based single-lead electrocardiogram device in detection of atrial fibrillation,” *Heart*, vol. 106, no. 9, pp. 665–670, 2020.
- [23] A. A. Hernandez, P. Bonizzi, R. Peeters, and J. Karel, “Continuous monitoring of acute myocardial infarction with a 3-lead ECG system,” *Biomedical Signal Processing and Control*, vol. 79, p. 104041, 2023.
- [24] J. W. Mason, E. W. Hancock, and L. S. Gettes, “Recommendations for the standardization and interpretation of the electrocardiogram,” *Circulation*, vol. 115, no. 10, pp. 1325–1332, 2007.
- [25] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, and A. Y. Ng, “Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network,” *Nature Medicine*, vol. 25, no. 1, pp. 65–69, 2019.
- [26] A. H. Ribeiro, M. H. Ribeiro, G. M. M. Paixão, D. M. Oliveira, P. R. Gomes, J. A. Canazart, M. P. S. Ferreira, C. R. Andersson, P. W. Macfarlane, W. Meira, T. B. Schön, and A. L. P. Ribeiro, “Automatic diagnosis of the 12-lead ECG using a deep neural network,” *Nature Communications*, vol. 11, p. 1760, 2020.

- [27] Z. I. Attia, S. Kapa, F. Lopez-Jimenez, P. M. McKie, D. J. Ladewig, G. Satam, P. A. Pellikka, M. Enriquez-Sarano, P. A. Noseworthy, T. M. Munger, S. J. Asirvatham, C. G. Scott, R. E. Carter, and P. A. Friedman, “Screening for cardiac contractile dysfunction using an artificial intelligence-enabled electrocardiogram,” *Nature Medicine*, vol. 25, no. 1, pp. 70–74, 2019.
- [28] P. Wagner, N. Strodthoff, R.-D. Bousseljot, D. Kreiseler, F. I. Lunze, W. Samek, and T. Schaeffter, “PTB-XL, a large publicly available electrocardiography dataset,” *Scientific Data*, vol. 7, p. 154, 2020.
- [29] S. Śmigielski, K. Pałczyński, and D. Ledziński, “ECG signal classification using deep learning techniques based on the PTB-XL dataset,” *Entropy*, vol. 23, no. 9, p. 1121, 2021.
- [30] D. Makowski, T. Pham, Z. J. Lau, J. C. Brammer, F. Lespinasse, H. Pham, C. Schölzel, and S. H. A. Chen, “NeuroKit2: A python toolbox for neurophysiological signal processing,” *Behavior Research Methods*, vol. 53, pp. 1689–1696, feb 2021.
- [31] H. He and E. A. Garcia, “Learning from imbalanced data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [32] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “Smote: synthetic minority over-sampling technique,” *J. Artif. Int. Res.*, vol. 16, p. 321–357, June 2002.
- [33] P. Sarker, K. Choi, A. A. Nahid, *et al.*, “Catboost with physics-based metaheuristics for thyroid cancer recurrence prediction,” *BioData Mining*, vol. 18, p. 84, 2025.
- [34] J. Di, C. Hickey, C. Bumgardner, M. Yousif, M. Zapata, T. Bocklage, B. Balzer, M. M. Bui, J. M. Gardner, L. Pantanowitz, and S. A. Qasem, “Utility of artificial intelligence in a binary classification of soft tissue tumors,” *Journal of Pathology Informatics*, vol. 15, p. 100368, 2024.
- [35] J. Sanjaya, S. Pathak, Y. Si, M. Haghi, N. Kudrot, G. Placencia, K. Alaei, and M. Pishgar, “Interpretable machine learning prediction of in-hospital mortality in icu patients with cancer and sepsis using first-day data: Development using mimic-iv and external validation in eicu-crd,” *medRxiv*, 2026.
- [36] D. Storcheus, A. Rostamizadeh, and S. Kumar, “A survey of modern questions and challenges in feature extraction,” in *Proceedings of the 1st International Workshop on Feature Extraction: Modern Questions and Challenges at NIPS 2015* (D. Storcheus, A. Rostamizadeh, and S. Kumar, eds.), vol. 44 of *Proceedings of Machine Learning Research*, (Montreal, Canada), pp. 1–18, PMLR, 11 Dec 2015.
- [37] A. Zheng and A. Casari, *Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists*. O’Reilly Media, Inc., Apr. 2018.
- [38] D. Sonnadara, T. T. Khuat, K. Musial-Gabrys, *et al.*, “Interpretable automated feature engineering: A comprehensive review with a focus on dynamic and stationary environments,” *Artificial Intelligence Review*, vol. 59, p. 165, 2026.
- [39] A. Mumuni and F. Mumuni, “Automated data processing and feature engineering for deep learning and big data applications: A survey,” *Journal of Information and Intelligence*, vol. 3, no. 2, pp. 113–153, 2025.
- [40] Y. Wu, F. Yang, Y. Liu, X. Zha, and S. Yuan, “A comparison of 1-d and 2-d deep convolutional neural networks in ecg classification,” *Conference proceedings : ... Annual International Conference of the IEEE Engineering in Medicine and Biology Society. IEEE Engineering in Medicine and Biology Society. Annual Conference*, vol. 2018, pp. 324–327,

2018.

- [41] H. Narotamo, M. Dias, R. Santos, A. V. Carreiro, H. Gamboa, and M. Silveira, “Deep learning for ecg classification: A comparative study of 1d and 2d representations and multimodal fusion approaches,” *Biomedical Signal Processing and Control*, vol. 93, p. 106141, 2024.
- [42] J. Li, Y. Si, T. Xu, and S. Jiang, “Deep convolutional neural network based ecg classification system using information fusion and one-hot encoding techniques,” *Mathematical Problems in Engineering*, vol. 2018, no. 1, p. 7354081, 2018.
- [43] L. van der Maaten, E. Postma, and H. Herik, “Dimensionality reduction: A comparative review,” *Journal of Machine Learning Research - JMLR*, vol. 10, 01 2007.
- [44] D. Peng, Z. Gui, and H. Wu, “Interpreting the curse of dimensionality from distance concentration and manifold effect,” 2025.
- [45] W. Jia, M. Sun, J. Lian, *et al.*, “Feature dimensionality reduction: A review,” *Complexity Intelligence Systems*, vol. 8, pp. 2663–2693, 2022.
- [46] K. Berahmand, F. Daneshfar, E. S. Salehi, *et al.*, “Autoencoders and their applications in machine learning: A survey,” *Artificial Intelligence Review*, vol. 57, p. 28, 2024.
- [47] W. Liu and I. W. Tsang, “Making decision trees feasible in ultrahigh feature and label dimensions,” *Journal of Machine Learning Research*, vol. 18, no. 81, pp. 1–36, 2017.
- [48] C. Lea, R. Vidal, A. Reiter, and G. D. Hager, “Temporal convolutional networks: A unified approach to action segmentation,” in *Computer Vision – ECCV 2016 Workshops*, vol. 9915 of *Lecture Notes in Computer Science*, (Cham), Springer, 2016.
- [49] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning* (F. Bach and D. Blei, eds.), vol. 37 of *Proceedings of Machine Learning Research*, (Lille, France), pp. 448–456, PMLR, 07–09 Jul 2015.
- [50] X. Ren, W. Wei, L. Xia, and C. Huang, “A comprehensive survey on self-supervised learning for recommendation,” *ACM Comput. Surv.*, vol. 58, Sept. 2025.
- [51] M. Bicego and F. Cicalese, “An interesting property of random forest distances with respect to the curse of dimensionality,” in *Structural, Syntactic, and Statistical Pattern Recognition: Joint IAPR International Workshops, S+SSPR 2024, Venice, Italy, September 9–10, 2024, Revised Selected Papers*, (Berlin, Heidelberg), p. 188–198, Springer-Verlag, 2024.
- [52] S. Si, H. Zhang, S. S. Keerthi, D. Mahajan, I. S. Dhillon, and C.-J. Hsieh, “Gradient boosted decision trees for high dimensional sparse output,” in *Proceedings of the 34th International Conference on Machine Learning* (D. Precup and Y. W. Teh, eds.), vol. 70 of *Proceedings of Machine Learning Research*, pp. 3182–3190, PMLR, 06–11 Aug 2017.
- [53] V. Syrgkanis and M. Zampetakis, “Estimation and inference with trees and forests in high dimensions,” in *Proceedings of Thirty Third Conference on Learning Theory* (J. Abernethy and S. Agarwal, eds.), vol. 125 of *Proceedings of Machine Learning Research*, pp. 3453–3454, PMLR, 09–12 Jul 2020.